

Enhancing Memory Bandwidth and Energy Efficiency in GPU Microarchitecture through High Bandwidth Memory (HBM) and Data Locality Optimizations

Suchibrata Das

Department of Electrical and Computer Engineering
Northeastern University, Boston, USA

The increasing demand for high-performance computing in fields such as artificial intelligence, data analytics, and scientific simulations has driven rapid advancements in GPU microarchitectures. GPUs, known for their massive parallelism and computational efficiency, face critical challenges in addressing the growing requirements of data-intensive workloads. Two of the most significant bottlenecks are memory bandwidth limitations and energy inefficiency, which hinder GPU scalability and performance. Modern GPUs rely on the Single Instruction Multiple Thread (SIMT) execution model, where thousands of threads operate concurrently. However, achieving peak performance requires efficient utilization of memory resources and optimal data access patterns. The integration of High Bandwidth Memory (HBM) has emerged as a promising solution to overcome bandwidth constraints, offering higher throughput and reduced energy consumption compared to traditional memory architectures. Simultaneously, architectural innovations aimed at improving data locality and mitigating control flow divergence are critical for maintaining energy efficiency and resource utilization.

This project explores the potential of HBM and data locality optimizations in enhancing GPU performance. By analyzing state-of-the-art research, the study aims to address the following objectives:

1. Evaluate the impact of HBM on memory bandwidth and processing speed.
2. Investigate data locality strategies to improve energy efficiency and reduce latency.
3. Identify and propose future directions for GPU microarchitecture optimization.

Through an extensive review of 15 research papers, this work highlights the techniques and methodologies that contribute to addressing these challenges. The discussion tries to compare existing approaches, identifies research gaps, and suggests innovative strategies to drive further advancements in GPU architecture.

GPU Architecture

Graphics Processing Units (GPUs) have revolutionized computing by enabling massive parallelism for data-intensive workloads. Their architecture is fundamentally different from CPUs, focusing on throughput rather than low-latency execution.

1. SIMT Execution Model:
 - GPUs operate on the Single-Instruction, Multiple-Thread (SIMT) model.
 - Threads are grouped into warps, with each warp containing 32 threads that execute instructions in lockstep.
 - Lockstep Execution: In SIMT model, threads within a warp execute the same instruction simultaneously, step by step, which is referred to as lockstep execution
 - Multiple warps are assigned to Streaming Multiprocessors (SMs), which handle instruction scheduling and execution.
2. Memory Hierarchy: Efficient memory management is critical for GPU performance. The memory hierarchy includes:
 - Registers: Fast, thread-specific storage for immediate data access.
 - Shared Memory: A low-latency memory shared by threads within a block.
 - Global Memory: High-capacity but high-latency memory accessible by all threads.
 - High Bandwidth Memory (HBM): Advanced memory technology providing significantly higher bandwidth compared to traditional GDDR.

3. Key Challenges:

- Control Flow Divergence: Threads in a warp may follow different execution paths, causing inefficiency.
- Memory Bandwidth Bottlenecks: Accessing global memory can cause high latency and stalls.
- Energy Consumption: GPUs require significant power to sustain their high levels of parallelism.

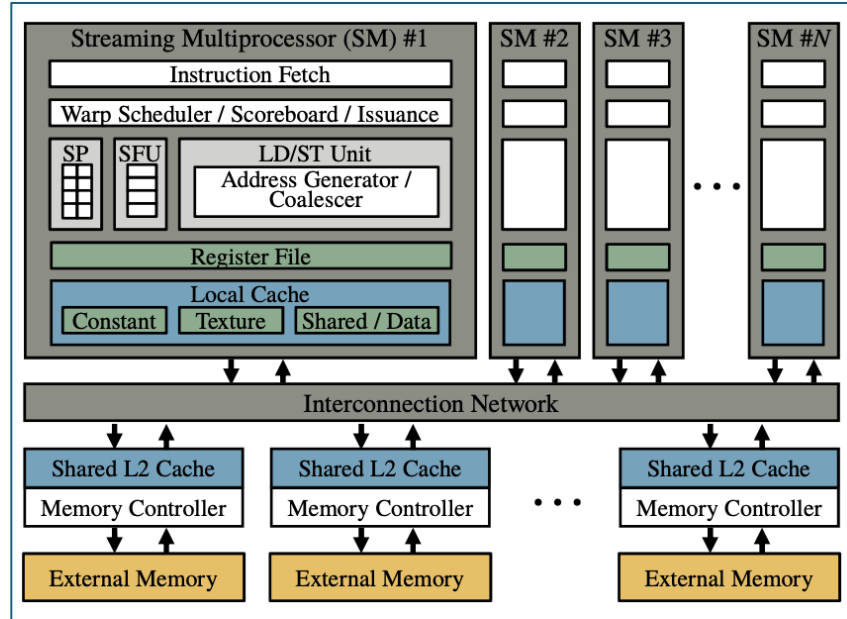


Figure 1- Baseline GPU architecture [4]

This figure shows the core components of a GPU, including Streaming Multiprocessors (SMs), memory hierarchy, and the interconnection network. It illustrates how data flows from external memory to SMs, highlighting the importance of efficient memory access and management.

Paper 1: Performance Evaluation and Optimization of HBM-Enabled GPU for Data-Intensive Applications

This paper evaluates the capabilities of High Bandwidth Memory (HBM)-enabled GPUs in managing data-intensive workloads. The focus is on two representative use cases: **Convolutional Neural Networks (CNNs)**, which have regular memory access patterns, and **Breadth-First Search (BFS)**, which is characterized by irregular memory access. The study highlights the potential of HBM in tackling modern computational demands while addressing its limitations through innovative optimizations.

Problem Statement: Despite offering remarkable bandwidth and energy efficiency, HBM-enabled GPUs face two main challenges: limited memory capacity (e.g., 4GB in AMD Radeon Fury X), which causes frequent memory transfers and latency, and inefficiency in handling irregular workloads like BFS due to scattered memory access patterns. These bottlenecks restrict HBM's effectiveness in data-intensive tasks like AI and graph analytics.

Proposed Solution: To address these challenges, the paper introduces workload-specific optimization techniques that significantly improve HBM utilization:

1. Optimizing CNN Training

- **Problem:** Training deep neural networks like AlexNet often exceeds the available HBM capacity, requiring frequent memory swaps.
- **Proposed Solution:**
 - A **software pipeline with double-buffering**:
 - Dynamically offloads unused neural network layers to CPU memory.
 - Prefetches the next required layers back to GPU memory during computation.
 - By overlapping data transfer and computation, this technique hides memory latency and ensures seamless execution.
- Results - Reduced memory consumption from **4.35GB to 3.52GB**.
 - Achieved a **1.63× speedup** in AlexNet training which is discussed in next section.

2. Optimizing BFS Traversal

- **Problem:** BFS workloads with irregular memory access patterns underutilize HBM bandwidth due to scattered accesses and imbalanced workloads.
- **Proposed Solutions:**
 1. **Warp-Centric Programming Model:**
 - Shifts from thread-centric to warp-centric mapping to coalesce memory accesses.
 - Balances workloads across threads within warps to improve bandwidth utilization.
 2. **Direction Optimization:**
 - Combines top-down and bottom-up graph traversal methods to minimize redundant memory operations.
 - Dynamically switches traversal directions based on graph structure to optimize performance for large-scale graphs.

Results: - Reduced clock cycles by **36%** through warp-centric programming. Achieved a **2.5× speedup** with direction optimization, resulting in a total **9.8× improvement** over traditional methods.

Methodology

The paper evaluates these strategies using:

1. AlexNet (CNN Training): The paper delves into the challenges and solutions of running AlexNet, a deep convolutional neural network, on HBM-enabled GPUs. AlexNet comprises five convolutional layers and requires significant computational power and memory. Key highlights include:

- **Oversubscription Issue:** Training AlexNet with larger batch sizes (e.g., 256) exceeds the 4GB memory limit of HBM, leading to data being offloaded to slower CPU memory. This incurs high latency due to bandwidth limitations of the PCIe bus.

- **Optimization through Software Pipelining:** The authors introduced a **double-buffering technique** to overlap computation with memory transfers. This approach effectively hides latency by transferring data asynchronously using two command queues. As a result, *memory consumption during training was reduced from 4.35GB to 3.52GB*
- **Figure 4** demonstrates the double-buffering pipeline used in CNN training, where two command queues handle computation and memory transfers in parallel. This synchronization mechanism overlaps data transfer and computation, reducing idle time and hiding memory latency
- **Results:** The proposed method achieved a **1.63× speedup** over conventional implementations on HBM-enabled GPUs, demonstrating the potential of software optimizations to mitigate hardware limitations

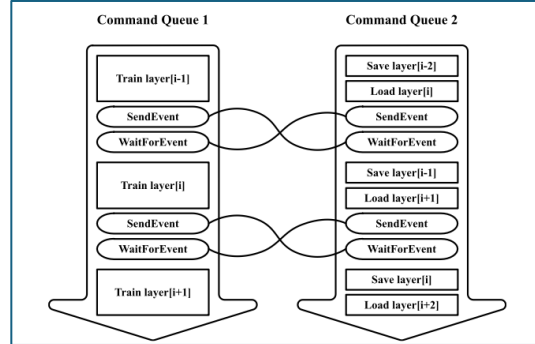


Figure 1: Synchronization and communication for parallel kernels in the command pipeline [1]

2. Breadth-First Search (Graph500 Benchmark): Breadth-First Search (BFS) is a widely used graph traversal algorithm in data-intensive applications. Its irregular memory access patterns pose challenges for HBM-enabled GPUs. Key contributions include:

- **Warp-Centric Programming Model:** Instead of mapping tasks to individual threads, the authors assigned tasks to warps, enabling coalesced memory access and better load balancing. This reduced clock cycles by **36%**
- **Direction Optimization:** The study combined top-down and bottom-up traversal strategies to minimize redundant memory accesses. This hybrid approach improved bandwidth utilization and achieved a **2.5× speedup** over warp-centric optimization alone.
- **Performance Gains:** Overall, the BFS implementation leveraging these techniques achieved up to a **9.8× speedup** compared to traditional thread-centric methods

Cycles	Warp 1		Warp 2		Warp 3		Warp 4	
	Thread 1	Thread 2	Thread 3	Thread 4	Thread 5	Thread 6	Thread 7	Thread 8
1	1->2	1->3						
2	1->4							
3	2->1	2->5	3->1		4->1	4->5		
4				IDLE	4->6	4->7		IDLE
5					4->8			
6	5->2	5->4	6->4		7->4	7->9	8->4	
7	9->7							

This figure illustrates how threads within warps cooperatively process graph edges (source vertex → destination vertex) over multiple cycles.

Figure 2- Time line of the warp-centric scheme. The entries are traversed edges denoted as source vertex pointing to destination vertex. [1]

This paper demonstrates the untapped potential of HBM-enabled GPUs through targeted software optimizations. By addressing capacity and access inefficiencies, it highlights how techniques like double-buffering, warp-centric programming, and direction optimization pave the way for leveraging HBM in next-generation AI and graph analytics workloads. These findings provide a foundation for optimizing GPU architectures to meet the evolving demands of data-intensive applications in modern computing environments.

Paper 2: Thread block compaction for efficient SIMT control flow

This paper proposes Thread Block Compaction (TBC), a novel mechanism to tackle the significant performance bottleneck of **control flow divergence** in GPUs. GPUs operate on the **Single-Instruction, Multiple-Thread (SIMT)** model, where threads in a warp execute instructions in lockstep. Divergence occurs when threads take different execution paths, leading to inefficient utilization of GPU resources. TBC introduces **block-wide resource sharing** and **thread compaction** to maximize warp efficiency, reduce memory traffic, and improve performance for applications with nested or irregular workloads.

Control flow divergence presents major challenges in SIMT architectures, especially in applications with:

- **Nested control flow:** When threads follow deeply nested and irregular paths, performance degrades.
- **Irregular workloads:** These workloads exhibit poor load balancing and underutilization of resources.
- **Dynamic warp formation:** Existing techniques like dynamic warp formation (DWF) introduce issues such as increased memory traffic and scheduling overhead.

Traditional mechanisms like **per-warp stacks** exacerbate these issues by isolating warp-level resources, resulting in fragmented and underutilized execution.

The paper aims to improve GPU efficiency by addressing these limitations with a new approach.

1. Block-Wide Stack

Traditional Mechanism: The traditional per-warp stack treats each warp independently, leading to fragmented execution paths and inefficient resource utilization. Each warp maintains its own stack for control flow, which becomes memory-intensive and leads to scheduling inefficiencies in divergence-heavy applications.

Proposed Mechanism: A block-wide stack allows warps within the same thread block to share resources, enabling better coordination at divergent branches.

- **Benefits:** Faster reconvergence, reduced memory overhead, and improved warp utilization.

2. Thread Compaction

At divergence, when threads within a warp take different paths at a branch, some threads become inactive, waiting for others to complete, reducing SIMD efficiency.

Compaction Mechanism: Threads from the same thread block are dynamically **reorganized into new warps** based on their execution paths.

- **For example:** Threads taking the same path are grouped into one warp. This ensures that only active threads are executed together, reducing idle execution units.
- **Benefits:** Improves **instruction throughput** by maximizing active threads. Ensures better use of GPU resources without introducing significant scheduling overhead.

3. Reconvergence

- **Traditional Mechanism:** Reconvergence at the warp level introduces idle cycles, delaying execution.
- **Proposed Approach:** The **block-wide stack** ensures that threads from different warps reconverge more efficiently at shared control points, minimizing resource contention and scheduling delays..

4. Warp Prioritization: Policies such as round-robin and age-based scheduling manage the execution of compacted warps to ensure fairness and efficiency.

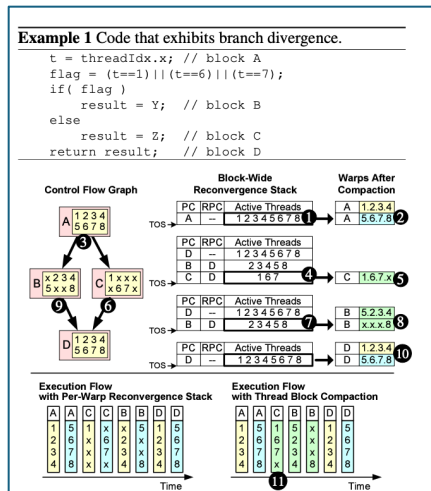


Figure 3 - High-level operation of thread block compaction. [2]

The Fig 3 shows how Thread Block Compaction (TBC) improves GPU performance by handling control flow divergence. Threads that take the same path (e.g., Block B or Block C) are grouped into new warps, so only active threads run together. The block-wide reconvergence stack helps manage and track threads, ensuring they rejoin efficiently. Compared to the traditional method (left), where many threads sit idle, TBC (right) reduces idle threads and makes better use of GPU resources. idle threads are represented by "x" in the execution flow section

The performance of TBC was evaluated on CUDA applications prone to control flow divergence using a modified **GPGPU-Sim** simulator. Key results include:

1. **Speedup:**
 - TBC achieves an average **22% speedup** over the baseline per-warp stack mechanism.
 - Demonstrates a **17% improvement** compared to Dynamic Warp Formation (DWF).
2. **Efficiency:**
 - Significant reduction in memory traffic.
 - Enhanced warp utilization through resource sharing and compaction.
3. **Real-World Impact:**
 - Applications with high divergence, such as those with irregular branches or deeply nested control flow, showed the greatest performance gains.

By mitigating control flow divergence, TBC demonstrates how GPU resource efficiency can be enhanced for a wide range of workloads. This innovation lays the foundation for future architectural improvements in SIMT designs.

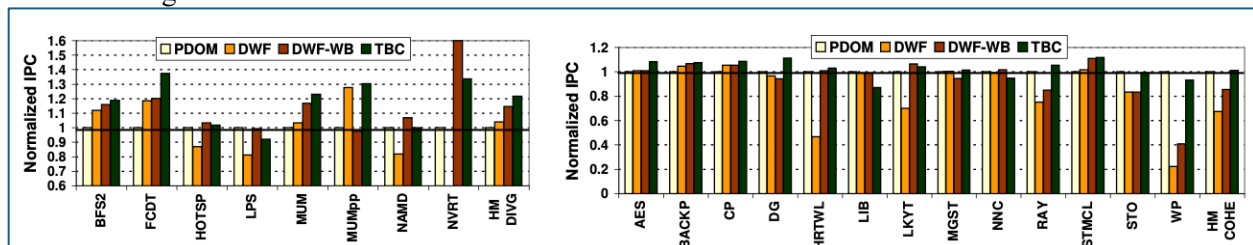


Figure 4 - Performance of Thread Block Compaction (TBC) and Dynamic Warp Formation (DWF) relative to baseline per-warp post-dominator reconvergence stack (PDOM) for the DIVG and COHE benchmark sets.

Fig 4 compares the performance of Thread Block Compaction (TBC) with Dynamic Warp Formation (DWF), Dynamic Warp Formation with Warp Barrier (DWF-WB), and the baseline Post-Dominator Reconvergence Stack (PDOM) across divergent (DIVG) and coherent (COHE) workloads. The left chart shows that TBC achieves the highest **normalized IPC** for divergence-heavy workloads, with an average **22% speedup** over PDOM and noticeable improvements over DWF and DWF-WB. The right chart demonstrates that TBC maintains comparable or slightly better performance for coherent workloads, confirming its efficiency across various application types. *This figure highlights TBC's ability to optimize GPU performance under challenging control flow*

Paper 3: A locality-aware memory hierarchy for energy-efficient GPU architectures

The paper highlights the **mismatch between GPU architecture and traditional memory hierarchies** which creates significant challenges for efficient performance. GPUs are built for massive parallelism, with thousands of threads running simultaneously, and depend on high memory throughput to keep their compute units active. However, traditional memory systems prioritize low-latency sequential access and rely on spatial and temporal locality, which often falls short for GPU workloads. This becomes a problem in applications like graph traversal and neural network training, where memory access patterns are irregular and fine-grained. As a result, GPUs face underutilized compute units, increased memory stalls, and higher energy consumption.

Locality-Aware Memory Architecture (LAMAR) adapts memory granularity, enhances data locality, and improves bandwidth usage to align memory performance with GPU needs, boosting both efficiency and speed.

Modern GPUs are designed to maximize parallelism but often suffer from inefficiencies due to poor memory locality. This leads to increased memory latency, suboptimal utilization of the memory bandwidth. And excessive energy consumption.

The paper identifies the root cause as the GPU's inability to adapt its memory hierarchy to irregular and fine-grained memory access patterns, which are common in many workloads.

Fine-Grained Access: Some applications work with small pieces of data spread across large memory areas. GPUs struggle to optimize these accesses because their memory hierarchy is built to handle large, contiguous chunks of data.

Proposed Solution: **LAMAR** is designed as a locality-aware memory hierarchy for GPUs that:

- Retains the advantages of coarse-grained accesses for programs with high spatial/temporal locality
- Allows selective fine-grained access to memory for data with low locality
- Adaptively adjusts the access granularity to reduce memory bandwidth and energy usage

LAMAR architecture includes:

Sub-ranked memory system: Allows fine-grained memory requests down to 32-byte granularity

Sector cache: Partitions each cache block into 4 32-byte sectors for fine-grained data management

Granularity Decision Unit (GDU): Determines the optimal access granularity for each cache miss

The paper proposes a **Bi-Modal Granularity Predictor (BGP)** that:

- Uses a bloom filter to track cache blocks with low/high locality
- Predicts whether to fetch data at coarse or fine granularity based on estimated spatial and temporal locality
- Adapts its default prediction based on observed access patterns

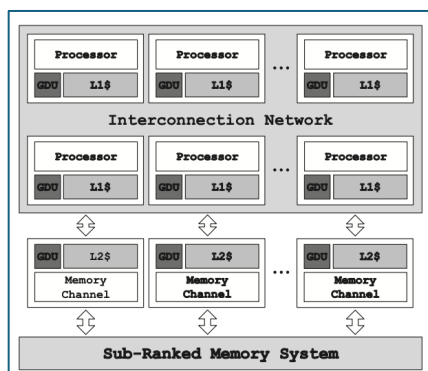


Figure 5: The proposed LAMAR GPU architecture. Each on-chip cache is sectored and is augmented with a GDU; off-chip memory is sub-ranked in order to allow fine-grained accesses. [3]

The experiments were conducted using a modified **GPGPU-Sim simulator** and focus on analyzing **performance improvements, memory bandwidth utilization, and energy efficiency**

1. CNN Training

Workload: Training of the AlexNet CNN model.

Environment: Caffe deep learning framework, modified to support OpenCL for GPU execution.

- **Memory Bandwidth:** LAMAR architecture efficiently utilized memory bandwidth, reducing stalls during CNN training caused by large-scale matrix operations and frequent weight updates.
- **Oversubscription Management:** Addressed HBM's capacity limitations by implementing a double-buffering pipeline, allowing efficient data transfer and avoiding delays during training.
- **Performance Improvement:** Maintained consistent training speeds despite reduced memory usage by overlapping computation with data transfer, ensuring smooth execution of large neural network models.

2. Breadth-First Search (BFS)

Setup

- **Workload:** Graph traversal using the BFS algorithm.
- **Environment:** **Graph500 benchmark**, a standard for evaluating graph traversal performance, generating large-scale random Kronecker graphs.
- **Load Imbalance:** Traditional thread-centric BFS approaches led to poor resource utilization due to irregular graph structures. LAMAR's **warp-centric policy** effectively addressed this by grouping threads based on similar tasks.
- **Coalesced Memory Access:** The warp-centric design optimized memory access by ensuring threads in a warp aligned with graph data structures, significantly increasing bandwidth utilization.
- **Performance:** Reduced execution cycles by 36%, showcasing substantial speedups for graph traversal in real-world low-diameter, scale-free graphs.

Performance Improvements: LAMAR improves GPU performance by up to 49% and on average 12-14% for fine-grained leaning applications compared to the baseline coarse-grained (CG) only memory system.

Memory Bandwidth Utilization: Significantly reduces off-chip traffic without degrading memory access behavior of coarse-grained leaning applications. Avoids over-fetching unnecessary data for irregular applications with low spatial/temporal locality.

Energy Efficiency: Improves energy efficiency for irregular applications that previously suffered from inefficient data fetching.

Cache Utilization: For 12 out of 20 benchmarks tested, more than 50% of L2 cache blocks were never reused before eviction in the baseline system.

Adaptive Granularity: Bi-Modal Granularity Predictor (BGP) adapts well to a variety of workloads and demonstrates high accuracy in predicting coarse or fine-grained accesses, improving cache hit rates and memory access efficiency. It provides robust benefits across a wide range of GPU applications

Paper 4 : Access pattern-aware cache management for improving data utilization in GPU

This paper addresses a critical challenge in GPU architectures: the inefficiencies in cache management caused by coarse-grained, warp-wide policies. Current strategies treat all load instructions within a warp uniformly, failing to account for diverse data locality behaviors. This limitation leads to poor cache utilization, increased contention, and degraded performance, particularly for memory-intensive applications. To overcome these challenges, the paper proposes Access Pattern-aware Cache Management (APCM) to improve data utilization in L1 caches - a fine-grained, load-specific approach that dynamically adapts cache management based on the data locality of individual load instructions.

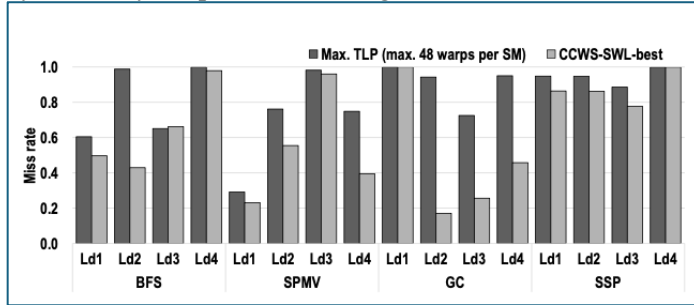


Figure 6: Miss rate change per load by warp throttling [4]

The figure [4] shows that **different load instructions (Ld1, Ld2, Ld3, Ld4)** exhibit varying degrees of cache miss rate improvement across benchmarks under warp throttling (**CCWS-SWL-best**) compared to maximum thread-level parallelism (**Max TLP**). In the BFS benchmark Ld2 showed a dramatic reduction in cache misses under warp throttling, while others (Ld1, Ld3, Ld4) experienced little benefit due to differing locality behaviors. These limitations underscore the need for a more granular, per-load approach.

Access Pattern Classification:

The authors categorize data locality into four types:

1. Streaming: Data used once.
2. Intra-warp locality: Data reused within the same warp.
3. Inter-warp locality: Data reused across different warps.
4. Mixed locality: A combination of intra- and inter-warp reuse.

Individual load instructions exhibit consistent locality patterns across all warps, enabling the application of load-specific cache policies.

Methodology

The proposed APCM dynamically detects the data locality of each load instruction using a monitor warp to track cache access patterns. Based on this classification, APCM applies:

Cache Bypassing: For streaming data, bypassing the L1 cache entirely to free up space for reusable data.

Cache Protection: For intra-warp locality, protecting cache lines to prevent premature eviction and maximize reuse. It estimates the lifespan of cached data

APCM introduces three key hardware components to implement fine-grained cache management: These components dynamically monitor access patterns and apply optimized cache management strategies tailored to each load instruction's behavior.

Monitor Tag Array (MTA): Tracks data sharing behavior for a monitored warp. Maintains counters and metadata for monitored warps, recording the number of cache hits, cache misses, and data sharing behaviors.

Cache Access Information Table (CAIT): Stores per-load cache management policies. Stores the cache management policies for each load instruction based on the classification provided by the MTA.

Protection Status Board (PSB): Maintains information about cache lines protected by specific warps. Tracks cache lines that are protected from eviction due to intra-warp or inter-warp locality. Maintains a list of cache lines that belong to protected load instructions. It prevents these cache lines from being replaced until their associated data locality has been fully exploited. Thus, ensures that reusable data remains in the cache for as long as it is beneficial, minimizing premature evictions.

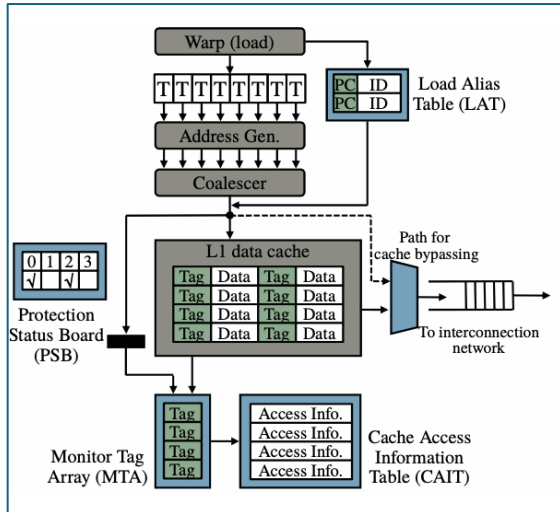


Figure 7 – Hardware architecture of APCM [4]

The workflow involves:

1. Monitoring data access patterns with the MTA.
2. Classifying each load instruction's data locality type.
3. Dynamically assigning cache management policies using the CAIT.
4. Enforcing protection for reusable cache lines via the PSB.
5. Continuously adapting policies as workloads evolve.

The effectiveness of APCM was evaluated using a modified **GPGPU-Sim** simulator with CUDA benchmarks. Experiments covered cache-sensitive, cache-moderate, and cache-insensitive workloads, focusing on performance, cache efficiency, and energy consumption.

Aspect	Results
Performance Improvement (IPC)	Average 22% improvement across all workloads; up to 34% for cache-sensitive workloads like BFS and MUM++.
Cache Miss Rate Reduction	15% reduction for cache-sensitive workloads (e.g., BFS, SPMV) and 22% for cache-moderate workloads (e.g., NAMD).
Energy Efficiency	27% energy savings due to improved cache utilization and fewer memory stalls.
Scalability	Effective across varying L1 cache sizes and warp scheduling policies (e.g., round-robin and greedy-warp)
Comparison with Baselines	Outperforms warp-wide methods like Per-Warp Reconvergence Stacks, Static Warp Throttling, and Adaptive Cache Bypassing.

Paper 5 - Analyzing CUDA Workloads Using a Detailed GPU Simulator

This paper investigates CUDA workloads using **GPGPU-Sim**, a detailed cycle-accurate GPU simulator. The authors analyze the impact of architectural choices on performance across a range of CUDA applications, highlighting bottlenecks, design trade-offs, and opportunities for optimization in GPU microarchitecture.

Key Methodology and Benchmarked Applications

- **Simulator Design:** GPGPU-Sim models NVIDIA's CUDA Parallel Thread Execution (PTX) and enables analysis of existing CUDA applications without source code modifications.

Applications Benchmarked: Twelve CUDA workloads spanning domains like graph algorithms (BFS, DG), encryption, molecular dynamics, and neural networks (NN).

Benchmark Characteristics: It provides detailed analysis of various CUDA applications, including:

- AES Encryption
- Breadth-First Search
- Coulombic Potential
- 3D Laplace Solver
- LIBOR Monte Carlo
- MUMmerGPU
- Neural Network Digit Recognition

Each benchmark is characterized by its grid and CTA dimensions, memory usage patterns, and specific performance bottlenecks.

- **Architectural Features Explored:** Interconnect topology, caching, branch divergence, memory controllers, memory coalescing, and DRAM access patterns.

Key Findings

1. Interconnect Topology and Bandwidth

- **Observation:** Applications like BFS and DG are highly sensitive to interconnect bandwidth but less affected by latency.
- **Result:** Doubling interconnect bandwidth improves performance by **21%**, especially for memory-intensive workloads.
- **Interconnect Contention:** Global memory accesses saturated bandwidth, emphasizing the importance of memory scheduling.

1. Caching Efficiency

Adding an **L1 cache** improved performance for memory-intensive workloads like BFS and NN, as it reduced DRAM requests. However, applications with high write-back traffic (e.g., CP and FWT) suffered from cache inefficiencies due to write misses and high eviction rates.

The **cache hit rate** varied significantly between workloads, showing that caching benefits depend on the memory access patterns.

The **texture cache** is a specialized GPU memory hierarchy component designed to improve performance for workloads with high **spatial locality**, such as image processing, 2D/3D rendering, or data with predictable access patterns.

Regular Workloads: Improved performance and reduced latency for predictable patterns like matrix multiplication or image convolution.

Irregular Workloads: Minimal benefits for scattered access patterns like BFS due to frequent cache misses.

Energy Efficiency: Reduced memory traffic for some workloads, improving energy efficiency by minimizing off-chip accesses.

- **Observation:** L1 and L2 caches improved memory-intensive workloads like BFS and NN by reducing DRAM requests.

- **Challenges:** Applications with high write-back traffic (e.g., CP, FWT) suffered from cache inefficiencies due to write misses and high eviction rates.

3. Branch Divergence

- **Challenge:** Divergent workloads like BFS suffered from increased memory stalls due to cache evictions and irregular access patterns.
- **Solution:** Efficient caching and coalescing reduced branch divergence penalties.

4. Memory Controller Design

- **Comparison:** FIFO vs. Out-of-Order (OoO) controllers.
- **Finding:** OoO controllers improved DRAM row buffer utilization and reduced memory latency, achieving **10-15% performance gains** for memory-bound workloads like BFS and DG.
- **Trade-off:** OoO controllers require additional hardware complexity but justify the cost for high-memory bandwidth applications.

5. Memory Coalescing

- **Techniques:**
 - **Intra-Warp Coalescing:** Combines memory requests from threads within a warp, improving bandwidth utilization.
 - **Inter-Warp Coalescing:** Merges requests from different warps for the same memory block.
- **Impact:** Intra-warp coalescing achieved a **6.1% speedup**, but irregular workloads like BFS showed limited gains due to non-contiguous access patterns.

6. DRAM Access Patterns

- **Observation:** Efficient utilization of DRAM bank-level parallelism (e.g., in MM) improved throughput.
- **Challenges:**
 - Irregular applications like BFS caused frequent DRAM row buffer conflicts, wasting bandwidth.
 - Fine-grained memory accesses in scattered workloads degraded memory throughput.

7. Thread Block and CTA Configuration

- **Tuning CTAs per Core:**
 - **Underutilization:** Too few CTAs reduced latency hiding.
 - **Overloading:** Too many CTAs caused contention for shared memory and register file resources.
- **Sweet Spot:** Optimal CTA configurations balanced throughput and resource usage.

Key Results

1. **Performance Gains:**
 - Doubling interconnect bandwidth resulted in a **21% performance improvement** for BFS and DG.
2. **Caching Benefits:**
 - L1 caches improved memory traffic and energy efficiency in NN and BFS, while write-heavy workloads (e.g., CP) suffered from cache inefficiencies.
3. **Divergence Management:**
 - Efficient caching and memory coalescing reduced branch divergence penalties in BFS.
4. **Memory Bandwidth Utilization:**
 - OoO controllers enhanced DRAM row buffer utilization, benefiting memory-bound applications.
5. **Energy Efficiency:**
 - Reduced memory traffic improved energy efficiency for regular workloads without increasing hardware complexity.

Paper 6: System-level Power Integrity Optimization Based on High-Density Capacitors for enabling HPC/AI applications

This research paper introduces platform-level power integrity (PI) solutions to enable high-power core IPs and high-bandwidth memory (HBM) interface for High-Performance Computing (HPC) and Artificial Intelligence (AI) applications. The study focuses on addressing the challenges of power integrity in systems requiring high-power operations exceeding 200 W.

Problem Statement

Modern HPC and AI accelerators, including GPUs, operate with high power densities and sharp current transients, leading to voltage drops, noise, and ripple in the power delivery network. These issues compromise system performance and reliability, posing a significant bottleneck in scaling compute-intensive architectures. Current PDN designs lack the efficiency and scalability to meet the rising demands of high-frequency, high-power systems

The HPC platform consists of multi-chip configurations, including a main processing chip and external memory chips, on a Si-interposer substrate. The design requires high-density interconnects between the System-on-Chip (SoC) and HBM chips to support wide bandwidth (>1 TB/s@HBM2)

1

Two main challenges in power and signal integrity (PSI) performance are:

1. Supporting high-power core IPs in a limited SoC area
2. Maintaining signal integrity for extremely large bandwidth operations

Proposed Solutions

1. High-Density Capacitor Integration:

- **Metal-Insulator-Metal (MIM) Capacitors:** These 20–40 nF/mm² capacitors are integrated directly on SoCs and interposers to provide localized decoupling, reducing voltage ripples near critical components.
- **Integrated Stacked Capacitors (ISC):** With a density of up to 300 nF/mm², ISC capacitors are deployed within interposers to enhance capacitance while maintaining a compact footprint.

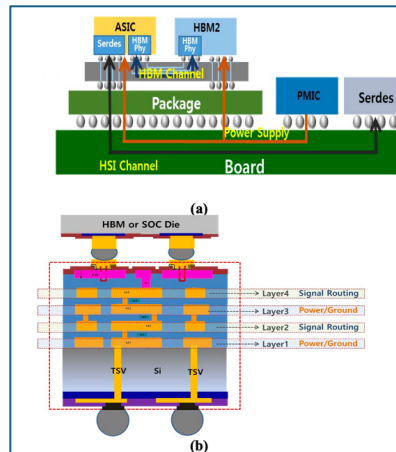


Figure 8 - (a) A schematic of 2.5D Si-interposer packaging structure and (b) a stack-up layout of Si-interposer [6]

2. Early-Stage PDN Optimization:

- A simulation-driven approach to PDN design evaluates capacitor placement and configuration to minimize impedance.
- Equivalent circuit models are utilized to predict the effectiveness of different capacitor layouts in real-world scenarios.

The researchers created an equivalent circuit model to analyze system-level Power Delivery Network (PDN) performance. The process involves:

- Extracting default capacitance and inductance values from physical layouts
- Estimating on-die capacitance and board inductance
- Performing what-if PI analysis during the initial design phase
- Optimizing PDN impedance by determining the optimal number and composition of decoupling capacitors

3. Scalability of Design:

- The proposed framework ensures minimal trade-offs between manufacturing cost and performance, making it applicable to a range of AI and HPC systems.

Results and Impact

The study applied high-density capacitor-based PI solutions to HPC/AI applications in the hundred-watt class, corresponding to a power density of 0.5 W/mm² in silicon size. Key findings include:

Impedance Reduction:

- PDN impedance between 10 MHz to 100 MHz reduced significantly, achieving a value as low as 0.002 in the ISC case, compared to the reference case without capacitors.

Voltage Drop Improvements in a 300W Test Case (Current Transition: 100 A/ns):

- **2-plate MIM capacitors:** 54% voltage drop reduction.
- **3-plate MIM capacitors:** 64% voltage drop reduction.
- **ISC capacitors:** 79% voltage drop reduction.

Voltage Drop Improvements in a 500W Test Case:

- **2-plate MIM capacitors:** 24% voltage drop reduction.
- **3-plate MIM capacitors:** 41% voltage drop reduction.
- **ISC capacitors:** 65% voltage drop reduction.

Impact of Current Transition Mitigation (From 100 A/ns to 5 A/ns):

- The voltage drop at the bump showed reduced sensitivity to the type of on-die capacitors, with consistent improvement levels maintained across configurations.

Conclusion

The proposed PI solution based on high-density capacitors offers significant improvements for high-power applications. The study emphasizes the importance of:

1. Adopting suitable on-die capacitors
2. Optimally combining with off-chip decoupling capacitors
3. Adjusting RTL architecture to mitigate current slope and maximum current levels

As applications requiring high power increase, initial PI analysis and design become crucial for preventing performance degradation in high-power designs

Paper 7: Bandwidth-Effective DRAM Cache: Improving the Efficiency of Large DRAM Cache Architectures

This paper addresses the inefficiencies in DRAM cache architectures, focusing on improving bandwidth utilization and memory access latency in high-performance computing (HPC) systems. As DRAM caches scale in size to meet the demands of HPC and data-intensive applications, they suffer from inefficiencies in bandwidth utilization due to tag lookup overheads and redundant memory accesses. These challenges limit the performance benefits of DRAM caches, especially for workloads with high memory bandwidth requirements.

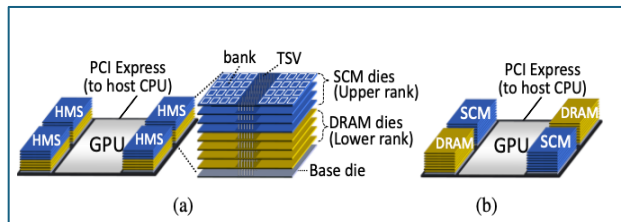


Figure 9 - Design space of a GPU with SCM and DRAM cache with (a) 3D-stacked DRAM and SCM and (b) separate DRAM and SCM stacks. [7]

The proposed DRAM cache design for GPUs with Storage-Class Memory (SCM) incorporates several novel features to maximize bandwidth efficiency and performance:

1. Aggregated Metadata-In-Last-column (AMIL)

- Co-locates all DRAM cache line tags in the last column's data portion of each row
- Allows fetching all tags in a row with a single column access, minimizing tag probe overhead
- Retains full ECC protection, unlike prior Tag-And-Data (TAD) organizations
- Uses the last column as it tends to be underutilized when data placement is aligned

2. SCM-aware Bypass Policy

- Considers multidimensional characteristics of GPU memory accesses and SCM properties
- Uses an SCM penalty score to account for spatial locality and write intensity
- Incorporates a DRAM-affinity score that combines the SCM penalty score with access frequency
- Bypasses DRAM cache for data with low performance utility
- Captures spatial locality, read/write access type, and access frequencies of pages

3. Configurable Tag Cache (CTC)

- Repurposes part of the L2 cache to store DRAM cache tags
- Allows flexible partitioning between L2 cache and tag cache capacity
- Reduces DRAM cache probe traffic with minimal cost overhead
- User-configurable for adaptability to different workloads

4. Large cache line size : Uses 256 B DRAM cachelines to achieve high memory bus utilization and exploit GPU spatial locality

5. Employs a direct-mapped organization to reduce tag size and enable efficient on-chip tag caching

Implements the design in a **Heterogeneous Memory Stack (HMS)** that stacks SCM dies on top of DRAM dies, as shown in figure. It shares memory channels between DRAM cache and SCM for flexible bandwidth utilization

This design effectively addresses GPU memory capacity limitations while maintaining high bandwidth utilization and performance for workloads that would otherwise suffer from oversubscription.

These innovations not only optimize bandwidth and reduce latency but also adapt dynamically to varying workload demands, ensuring robustness across diverse GPU applications

- Performance Improvements:** Compared to HBM, the proposed Heterogeneous Memory Stack (HMS) improves performance by up to $12.5x$ ($2.9x$ overall). As shown in Fig. 18, the runtime performance of the proposed design outperforms baseline configurations, validating the efficiency of CTC ways in minimizing latency
- The HMS reduces energy consumption by up to 89.3% (48.1% overall) compared to HBM. Fig. 19 illustrates the significant energy savings achieved with HMS, showcasing a reduction of up to 89.3% compared to HBM.

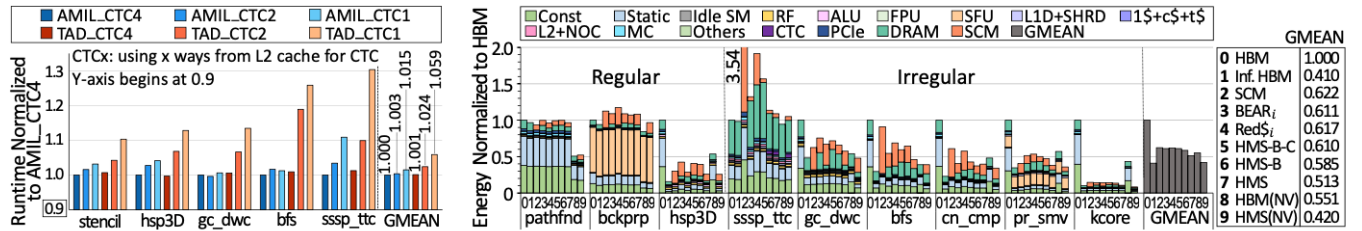


Figure 10. Performance impact of CTC ways [7] Figure 11. Energy consumption with different memory designs normalized to HBM [7]

- Traffic Reduction:** The proposed design reduces DRAM cache probe traffic by 91-93% compared to prior works.
 SCM write traffic is reduced by 57-75% compared to previous approaches.
- Effectiveness for Different Configurations:** The proposed DRAM cache design is effective even when SCM is integrated as separate devices or attached with high-speed links.
- Thermal Management:** Even without throttling, the worst-case peak HMS temperature differs from that of HBM by less than 0.1%.
 Simple SCM throttling techniques can effectively mitigate thermal issues when necessary.

The proposed DRAM cache design for GPUs, integrating innovations such as AMIL, SCM-aware bypass policy, and configurable tag cache, significantly enhances bandwidth utilization and energy efficiency. By leveraging the Heterogeneous Memory Stack (HMS) architecture, this work bridges the gap between traditional HBM and the growing demands of HPC and AI workloads. The experimental results validate the design's effectiveness in improving performance by up to $12.5x$ and reducing energy consumption by 89.3%. This study underscores the potential of heterogeneous memory architectures to redefine GPU scalability, offering a robust foundation for future advancements in memory hierarchy design.

Paper 8: Optimizing Bandwidth and Power of Graphics Memory with Hybrid Memory Technologies and Adaptive Data Migration

This paper explores hybrid memory architectures for GPUs, focusing on leveraging DRAM, STT-RAM, and RRAM technologies to optimize memory bandwidth and reduce power consumption. GPUs face increasing challenges with memory bandwidth bottlenecks and rising power demands as computational capabilities scale.

Hybrid Memory Architecture

The authors propose a hybrid graphics memory architecture that combines different memory technologies. It is partitioned into three distinct regions based on the characteristics of the memory technologies and the workload's access patterns:

- **DRAM (Dynamic Random-Access Memory):**
 - **Primary Role:** Handles frequently accessed data (hot data) that requires high bandwidth.
 - **Why DRAM?** DRAM is fast and offers high throughput, making it ideal for data-intensive tasks where latency is critical.
 - **Optimization in the Design:** By reducing the DRAM size, the system can operate at higher clock frequencies, improving memory bandwidth utilization by up to 25%.
- **STT-RAM (Spin-Transfer Torque RAM):**
 - **Primary Role:** Used for infrequent but high-intensity writes.
 - **Why STT-RAM?** It has high endurance for writes compared to other NVMs (non-volatile memories) and can support write-heavy tasks without degrading performance or memory lifetime.
- **RRAM (Resistive RAM):**
 - **Primary Role:** Stores read-only or low-write-intensity data.
 - **Why RRAM?** Its low power consumption and ability to retain data without frequent refresh cycles make it suitable for less dynamic data. It excels in energy efficiency and is perfect for storing static data or read-heavy workloads.

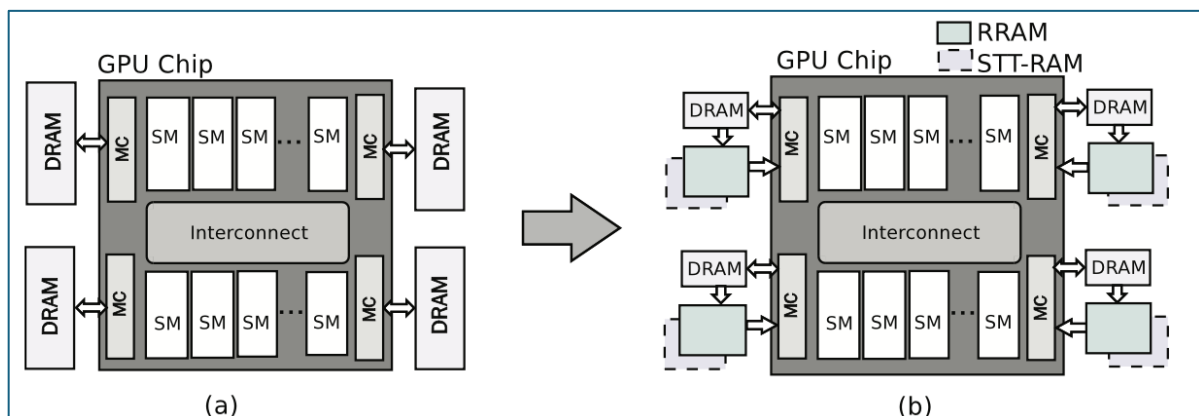


Figure 12 - Overview of GPU system with hybrid memory. (a) Conventional GPU system with off-chip GDDR6s. (b) GPU system with hybrid memory [8]

Reduced DRAM Capacity: Halving DRAM size improves bandwidth by up to 25% by enabling higher clock frequencies.

Dynamic Migration Process: Data is **dynamically reallocated** to appropriate memory tiers based on access frequency, read/write patterns, and spatial/temporal locality.

The process minimizes unnecessary movement to avoid: Excessive wear on NVMs like STT-RAM and RRAM. Performance overhead due to frequent migrations.

Energy-Aware Partitioning: Data placement reduces energy consumption by **prioritizing DRAM for hot data** while offloading less critical data to NVMs. This leads to **better bandwidth utilization** without the need for increasing memory capacity.

Workload Scenario: A **large-scale neural network training** workload with multiple layers and frequent weight updates:

- **Hot Data:** Layers being actively trained are kept in DRAM, ensuring fast access to weights and gradients.
- **Idle Layers:** Weights of layers not currently being trained are moved to **STT-RAM**, which handles intermittent writes efficiently.
- **Static Metadata:** Configuration data and static parameters are stored in **RRAM**, minimizing energy usage for read-only access.

In this example, the **adaptive migration mechanism** ensures: **Fast access** to actively used data in DRAM. **Efficient handling** of sporadic writes in STT-RAM. **Minimal energy use** for static data in RRAM.

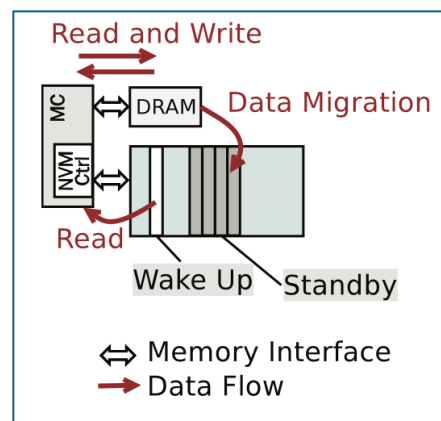


Figure 13 - Memory interface configuration and data flow between memory controller and hybrid memory.[8]

The figure illustrates the hybrid memory architecture integrating DRAM and Non-Volatile Memory (NVM) with a memory controller. Frequently accessed data is stored in DRAM for low latency, while less-used data resides in energy-efficient NVM. The memory controller dynamically manages **data migration**, optimizing bandwidth and energy usage. NVM operates in **active and standby states**, reducing power consumption during idle periods. This architecture effectively balances performance and energy efficiency, addressing the needs of high-bandwidth and energy-sensitive workloads.

Performance Evaluation - System Throughput: Achieved a 12% improvement in instructions executed per second compared to baseline GDDR5 memory.

Memory Bandwidth: Halving DRAM capacity and increasing clock frequency boosted bandwidth by up to 25%.

Power Consumption Analysis: Memory Power: Hybrid memory design reduced memory power usage by an average of 31%.

System Power: Overall GPU system power consumption dropped by 16%.

Power Breakdown: Detailed energy profiling across DRAM, STT-RAM, and RRAM highlighted application-specific savings.

Energy Efficiency : The hybrid memory showed a 33% increase in energy efficiency (measured as instructions per second per Watt) across all tested applications.

Application-Specific Insights: Benchmarked 20 GPU applications from NVIDIA CUDA SDK, Rodinia, and GPGPU-sim. Categorized workloads based on bandwidth demands, highlighting suitability for various hybrid memory configurations.

Paper 9: Performance evaluation of High Bandwidth Memory for HPC Workloads

The paper focuses on analyzing the performance benefits and trade-offs of integrating High Bandwidth Memory (HBM) in High-Performance Computing (HPC) systems. HBM stacks multiple DRAM dies vertically, connecting them through TSVs, allowing for a smaller physical footprint and higher data transfer rates. Unlike conventional DDR or GDDR memory, HBM achieves higher bandwidth while consuming significantly less energy per bit. The vertical stacking and proximity to processing units (e.g., CPUs or GPUs) reduce latency, making it highly suitable for data-intensive applications. **Advantages of HBM:**

- **High Bandwidth:** Offers up to 1 TB/s bandwidth, addressing the limitations of conventional memory.
- **Low latency:** Proximity to processing units and TSVs allow for faster data access, enhancing latency-sensitive workloads.
- **Enhanced parallelism:** Supports multiple independent channels and pseudo-channel architectures, enabling efficient simultaneous memory accesses.
- **Compact Design:** Smaller memory footprint due to 3D stacking.
- **Power Efficiency:** Consumes less power per unit of bandwidth, offering better energy efficiency.

HPC systems face unique memory bottlenecks due to the data-intensive nature of workloads, including simulations, machine learning, and graph analytics. Traditional memory technologies fail to meet these growing bandwidth demands, leading to performance limitations in modern HPC applications.

Challenges Highlighted:

- **Bandwidth Bottlenecks:** Applications with high memory throughput requirements often underperform on DDR or GDDR due to limited bandwidth.
- **Diverse Workload Characteristics:** HPC workloads differ significantly in their memory access patterns (e.g., streaming vs. random access), making it difficult to design a one-size-fits-all solution.
- **Energy-Performance Trade-offs:** While HBM improves energy efficiency, the power-to-cost ratio is a concern for large-scale deployment.
- **Integration Complexity:** Adapting existing hardware architectures to support HBM presents design and compatibility challenges.

The paper conducts two main types of experiments to evaluate the performance of High Bandwidth Memory (HBM2) compared to GDDR5 and GDDR6 technologies:

Synthetic Benchmark Tests

The researchers used synthetic benchmarks to stress test the memory systems under controlled conditions:

1. **Read and Write Latency Tests:**
 - Varied the number of read and write instructions from 500 to 5000
 - Compared average and 99th percentile latencies for HBM2 and GDDR6
2. **Read/Write Ratio Tests:**
 - Evaluated performance with different read-to-write ratios (0.33 and 2)
3. **Page Policy Comparison:**
 - Compared open and closed page policies for HBM2

Results:

- HBM2 significantly outperformed GDDR6 in read latency:
 - For 500 instructions: HBM2 averaged 52.8 cycles vs 983.0 cycles for GDDR6
 - HBM2 latency remained stable (52.8 to 55.2 cycles) as instructions increased from 500 to 5000
 - GDDR6 latency increased from 983.0 to 1483.1 cycles
- HBM2 also performed better in write latency:
 - For 500 instructions: HBM2 averaged 730.1 cycles vs 1003.8 cycles for GDDR6
- Closed page policy performed better for HBM2 with synthetic benchmarks

Real-World HPC Benchmark Tests

The researchers used High-Performance Computing (HPC) workloads from the Rodinia Benchmarks Suite to evaluate real-world performance:

1. **Execution Time Comparison:** Measured normalized execution time across different benchmarks
2. **Memory Cycles Analysis:** Compared the number of memory cycles spent for each benchmark
3. **Energy Efficiency Evaluation:** Analyzed energy consumption for different benchmarks

Key Findings:

- Execution time: HBM2 provided ~30% improvement compared to GDDR6 across benchmarks

These improvements are clearly illustrated in Fig. x, where HBM2 consistently outperforms GDDR5 and GDDR6 in normalized execution time, with the SRAD benchmark demonstrating the most significant gains.

Largest improvement: HBM2 consistently achieves better normalized execution times across various benchmarks compared to GDDR5 and GDDR6, as illustrated in Fig. 8, with SRAD showing the most significant improvement of 52.45%.

- Memory cycles: HBM2 spent significantly less time in memory

Example: 34.38% less time than GDDR5 and 23.47% less than GDDR6 for backpropagation benchmark

- Energy efficiency: HBM2 achieved ~23% energy savings compared to GDDR6

Largest improvement: 28.13% more energy-efficient for myocyte benchmark

Overall, the experiments demonstrated that HBM2 offers substantial performance improvements and energy savings compared to GDDR5 and GDDR6 for both synthetic and real-world HPC workloads, primarily due to its high bandwidth, low latency, and efficient 3D stacked architecture.

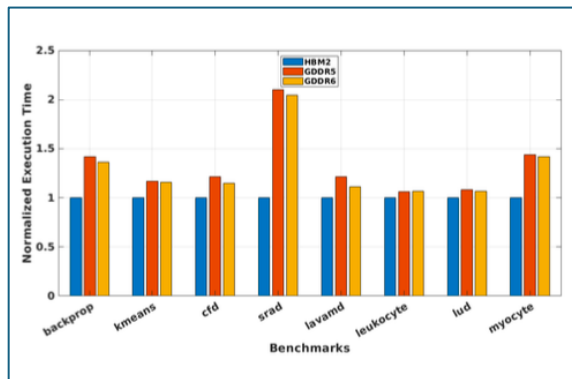


Figure 14: Normalized execution time spent in executing different benchmarks with HBM2, GDDR5 & GDDR6 [9]

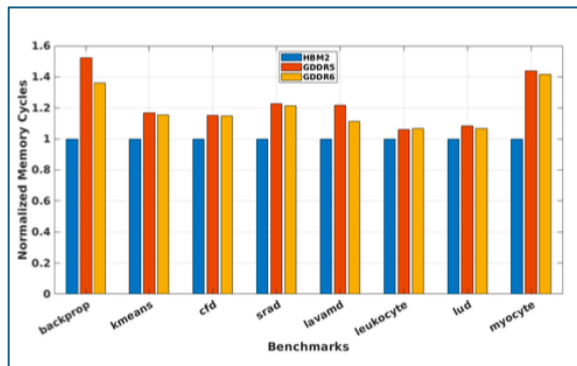


Figure 15: Normalized memory cycles in executing different benchmarks with HBM2, GDDR5 and GDDR6 [9]

Paper 10: Efficient Parallel Graph Exploration on Multi-Core CPU and GPU

Background: A **graph algorithm** is a computational procedure designed to solve problems that involve graph data structures. A **graph** consists of a set of **nodes** (also called vertices) connected by **edges**, which may represent relationships, connections, or interactions between entities. Graph algorithms are used to process, analyze, and solve problems related to such structures. **Graph algorithms** on GPUs leverage parallel processing to accelerate tasks like traversal, shortest path finding, and clustering by efficiently handling the irregular memory access and massive data parallelism inherent in graph structures. Graph algorithms are fundamental in various domains such as intelligence analysis, robotics, social network analysis, and computational biology. These algorithms often process massive datasets, which traditionally required long processing times. Parallelism has been challenging to apply effectively to graph algorithms due to their random memory access patterns.

This paper introduces methodologies for optimizing **graph exploration** algorithms, particularly **Breadth-First Search (BFS)**, on heterogeneous systems using both CPUs and GPUs. The methodology focuses on efficiently handling the unique challenges of graph exploration, such as irregular memory access patterns, workload imbalance, and scalability across architectures.

This paper focuses on graph algorithms, particularly breadth-first search (BFS), in the context of parallel processing on multi-core CPUs and GPUs. Breadth-first search (BFS) is highlighted as one of the most important graph algorithms for several reasons:

- It serves as a building block for many other algorithms
- It's used in betweenness centrality calculation, connected component identification, community structure detection, and max-flow computation
- It's commonly included in benchmark suites targeting graph applications

The paper presents a level-synchronous parallel BFS algorithm, which visits all nodes in each BFS level in parallel, with synchronization at the end of each level iteration. The authors note that real-world graphs often exhibit the "small world phenomenon," meaning they have small diameters even for large graph instances. This property makes the level-synchronous BFS algorithm particularly effective.

Hybrid CPU-GPU Method

To address potential inefficiencies with small graphs or high-diameter graphs, the authors propose a hybrid approach that dynamically selects the best execution method for each BFS level:

- Sequential execution
- Queue-based parallel CPU execution
- Read-based parallel CPU execution
- GPU execution

Queue-Based BFS (CPU):

- Utilizes shared queues to manage BFS frontiers. A frontier, in BFS, represents the set of nodes currently being processed in a given traversal level
- Employs local queues and ticket-lock mechanisms to reduce contention among threads and optimize memory access.

Read-Based BFS (CPU):

- Replaces queues with an array that tracks BFS levels, reducing overhead and optimizing memory bandwidth.

Hybrid BFS (CPU):

- Dynamically switches between Queue-Based and Read-Based BFS, depending on the graph size and BFS frontier at each level.

Hybrid CPU-GPU Execution

Dynamic Selection of Resources: Initial BFS levels (small frontiers)- Executed on the CPU due to low parallelism. Larger BFS levels (large frontiers)- Transitioned to the GPU to leverage its massive parallelism.

Efficient Data Transfers: Only the BFS frontier is transferred between CPU and GPU to minimize PCIe communication overhead.

Load Balancing: Dynamically distributes workloads between CPU and GPU based on the graph structure and frontier size.

3. Graph Benchmarks - Graph Types:

Uniform Random Graphs (Erdős-Rényi): Balanced degree distributions for evaluation of uniform workloads.

Scale-Free Graphs (RMAT): Skewed degree distributions reflecting real-world graph structures.

Benchmarks include graph sizes with varying diameters and degrees to evaluate adaptability.

4. Optimization Techniques

Memory Access Optimizations:

- Uses coalesced memory accesses on the GPU to improve bandwidth utilization.
- Local caching and atomic operations on the CPU reduce contention during shared memory access.

Frontier Size Adaptation:

- Dynamically adapts BFS execution strategy based on frontier size (CPU for small, GPU for large).

Thread Block Tuning:

- Optimizes thread block size and cooperative thread array (CTA) configurations on the GPU for efficient resource utilization.

Experiment Aspect	Implementation/Setup	Findings
CPU BFS Implementations	Queue-Based, Read-Based, and Hybrid BFS implementations tested on multi-core CPUs.	Read-Based BFS improved performance by up to 45% over Queue-Based BFS for large graphs.
Hybrid CPU-GPU Execution	Dynamic distribution of BFS frontiers: small frontiers on CPU, large frontiers on GPU.	Hybrid CPU-GPU approach achieved 2-3x speedup over CPU-only execution for large graphs.
Graph Characteristics	Benchmarked on uniform random graphs (Erdős-Rényi) and scale-free graphs (RMAT) with varying diameters.	Small-diameter graphs benefited from GPU parallelism; high-diameter graphs CPUs performed
Memory Access Optimization	Optimized memory access using coalescing for GPUs and local caching for CPUs.	Coalesced memory accesses enhanced bandwidth utilization; caching reduced contention on CPUs.
Dynamic Workload Balancing	Adaptation of execution strategy based on BFS frontier size and workload characteristics.	Hybrid execution prevented performance cliffs, balancing workloads across graph sizes and types.
Energy Efficiency	Reduced memory traffic and selective data transfers improved energy efficiency.	Optimized execution reduced memory traffic, improving energy efficiency

Paper 11: Efficiently Using a CUDA-enabled GPU as a Shared Resource

This paper focuses on overcoming the limitations of NVIDIA's CUDA-enabled GPUs, particularly their inability to run multiple kernels concurrently or share processing power efficiently among various processes. The authors propose a novel GPGPU service that enables the GPU to act as a shared resource, allowing multiple clients to utilize its processing capabilities simultaneously without a loss in performance.

Problem Statement:

- CUDA-enabled GPUs traditionally cannot run functions concurrently due to the nature of their SIMT (Single Instruction, Multiple Threads) architecture.
- GPUs are generally utilized exclusively by a single process, making them inefficient for workloads that do not fully utilize the GPU's massive parallel processing capabilities.
- Tasks with limited data parallelism or unknown runtime characteristics struggle to leverage GPUs effectively.

Proposed Solution: The paper introduces a **GPGPU Service** that:

1. **Persistent Kernel Execution:** Launches an infinite-running kernel that provides multiple functions.
2. **Block-level Control:** Each block in the kernel decides which function to execute based on memory transfers.
3. **Asynchronous Memory Transfers:** Enables seamless data exchange between the host and the GPU during kernel execution.
4. **Priority Scheduling:** Allows high-priority tasks to be allocated GPU resources dynamically.
5. **Block Reservation:** Ensures predictable response times for critical tasks.

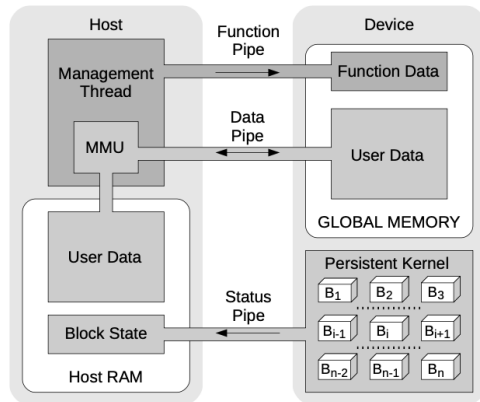


Figure 16: GPGPU service, coarse grained [11]

The above figure shows the architecture of the proposed GPGPU service. It allows efficient sharing of a CUDA-enabled GPU among multiple clients. The design introduces a management thread on the host, which communicates with the GPU device through three primary data pipelines: the **Function Pipe**, **Data Pipe**, and **Status Pipe**.

The **Management Thread** on the host coordinates task scheduling and memory management, interfacing with a dedicated **Memory Management Unit (MMU)** to optimize data transfers between the host RAM and the GPU's global memory. The **Persistent Kernel** running on the GPU is divided into multiple blocks ($B_1, B_2, \dots, B_n$), where each block dynamically fetches its assigned function and data from the **Function Pipe** and executes it.

Status updates are communicated back to the host via the **Status Pipe**, ensuring real-time monitoring and block-level task state management. The **Global Memory** stores both user data and the function data required for execution, enabling asynchronous memory operations for increased performance.

Key Innovations:

- **Memory Management Unit (MMU):** A dedicated unit that synchronizes data between the host and GPU and ensures efficient memory usage during kernel execution.
- **Dynamic Function Assignment:** Blocks fetch their function identifiers from memory, execute the assigned function, and report their status back to the host.
- **Flexibility for Diverse Applications:** The approach supports workloads like web services and terminal servers, which previously could not efficiently utilize GPUs.

Experiments and Results:

1. Test Setup:

- Hardware: GTX260 and GTX280 GPUs.
- Tasks: Sorting of sequences and matrix multiplication.
- Parameters: Utilization levels and the ratio of sorting to multiplication requests.

2. Findings:

- **Sorting Performance:** The GPGPU service achieved significantly higher sorting rates compared to conventional GPU approaches, particularly under high utilization.
- **Multiplication Performance:** Slightly lower performance (95%) due to service overhead.
- **Resource Sharing:** Total performance increased when tasks with different resource demands (e.g., sorting vs. multiplication) were processed simultaneously.

3. Performance Scaling:

- The service scaled linearly with utilization, ensuring consistent per-block performance across varying workloads.

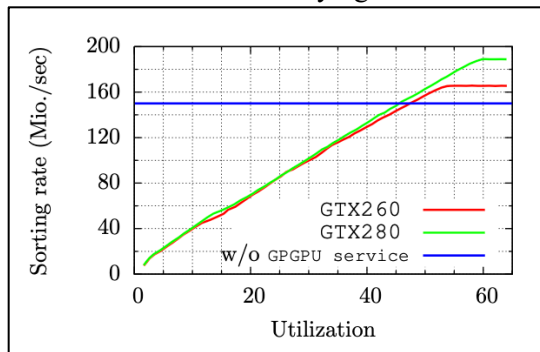


Figure 17 - Sorting rates [11]

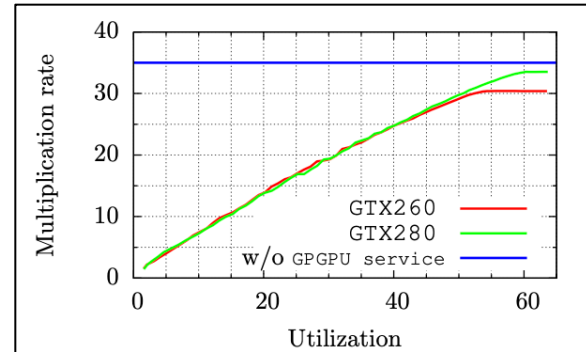


Figure 18 - Multiplication rates [11]

This graph in figure 17 illustrates the sorting rate (Mio./sec) for two GPUs (GTX260 and GTX280) using the GPGPU service compared to the baseline without the service. The sorting rate increases linearly with GPU utilization, exceeding the baseline after 47% utilization. This improvement highlights the efficiency of the GPGPU service in reducing overhead from kernel launches and synchronization for sorting tasks.

This graph in figure 18 shows the multiplication rate (Mio./sec) for the same GPUs. Unlike sorting, the multiplication rate with the GPGPU service is slightly lower (95% of the baseline) due to management overhead. The linear scaling with utilization indicates that the service effectively distributes workloads, but the lack of synchronization benefits limits performance gains for highly parallel matrix multiplication tasks.

Paper 12: Combining HW/SW Mechanisms to Improve NUMA Performance of Multi-GPU Systems

This paper presents a comprehensive study on improving NUMA (Non-Uniform Memory Access) performance in multi-GPU systems through a combination of hardware and software mechanisms.

Present challenges in NUMA:

- Multi-GPU systems offer increased computational resources but suffer from NUMA bottlenecks due to limited inter-GPU bandwidth compared to local GPU memory bandwidth.
- Existing software techniques like page migration and replication have limitations in addressing NUMA bottlenecks, especially for read-write shared pages.
- Caching remote data in GPU last-level caches (LLCs) is ineffective due to large shared working set sizes exceeding LLC capacity.

This paper investigates the combined use of state-of-the-art hardware and software mechanisms to overcome these limitations. It introduces **Caching Remote Data in Video Memory (CARVE)**. It is a novel hardware mechanism that addresses these challenges by reserving a small portion of GPU memory as a cache for remote shared data. By transforming GPU memory into a hybrid structure, CARVE combines high-speed access to remote data with efficient memory utilization, significantly improving NUMA performance.

It uses a small part of a GPU's memory as a special storage area to keep copies of data from other GPUs. This helps the GPU access shared data much faster because it doesn't need to rely on slower connections between GPUs. Instead of always fetching data from other GPUs, CARVE allows the GPU to store and reuse the data locally.

Components of CARVE and how it works:

1. **Remote Data Cache (RDC):**

CARVE sets aside a fraction of the GPU's video memory (e.g., 3%) as a dedicated cache for storing frequently accessed remote data. This cache works entirely in hardware and is invisible to the user or the operating system.

2. **Cache Management:**

When a GPU requests data:

- **Cache Hit:** If the data is already in the RDC, the GPU accesses it locally, avoiding delays from remote memory fetches.
- **Cache Miss:** If the data isn't in the cache, it is fetched from the remote GPU, stored in the RDC, and then used.

3. **Fine-Grained Caching:**

CARVE stores data in smaller units (e.g., 128-byte cachelines) rather than full memory pages. This reduces false sharing, where multiple GPUs unnecessarily share large memory blocks.

4. **Cache Coherence:**

To ensure consistency across GPUs, CARVE employs hardware coherence mechanisms:

- An **Epoch Counter** marks whether the cached data is still valid or needs to be updated.
- A **write-through policy** ensures that any changes to data are immediately propagated to remote GPUs, avoiding stale data in the cache.

5. **Hybrid Memory Structure:**

CARVE transforms GPU memory into a hybrid system, where part of the memory is used as regular memory, and part functions as a high-speed cache for remote data. This design ensures efficient use of memory resources while significantly improving performance.

Highlights on the memory structure of CARVE:

CARVE introduces a hybrid memory design that divides the GPU memory into two functional regions: one for standard application memory usage and another as a dedicated cache for remote data. This design ensures efficient utilization of GPU resources while addressing NUMA bottlenecks.

- **Dual-Purpose Functionality:**
By reserving a small portion of the GPU memory (e.g., 3%) as a **Remote Data Cache (RDC)**, CARVE enables GPUs to handle both local computations and remote data caching without sacrificing memory availability for applications.
- **Efficient Memory Access:**
The RDC allows frequently accessed remote data to be stored locally, transforming remote memory requests into high-speed local accesses. This drastically reduces the performance impact of low inter-GPU bandwidth.
- **Minimal Overhead:**
Despite reserving a fraction of the GPU memory for caching, CARVE ensures that this allocation has a negligible effect on the GPU's primary memory usage. Any potential loss of GPU memory capacity is mitigated by leveraging **Unified Memory (UM)**, which allows pages to be migrated between GPU and system memory as needed.
- **Key Advantage:**
This hybrid approach strikes a balance between improving performance and maintaining the memory required for application tasks. It ensures that the GPUs can handle large workloads while significantly reducing delays caused by NUMA bottlenecks.

Experimental Setup

- **System Configuration:**
 - A 4-node multi-GPU system was used for evaluations, where each GPU has 32GB of memory and is connected through inter-GPU links (e.g., NVLink).
 - Benchmarks were drawn from various workloads, including high-performance computing (HPC), machine learning (ML), and data-intensive applications.

The performance of CARVE was compared to:

Baseline NUMA-GPU: A traditional system using first-touch memory mapping and caching in GPU LLC.

NUMA-GPU with Software Paging: Using software to replicate read-only shared pages locally.

Ideal NUMA-GPU System: A hypothetical system that avoids all NUMA bottlenecks by satisfying all memory requests locally.

Configuration	Slowdown (%)	Remote Memory Accesses (%)	Memory Overhead (%)	Performance Gap to Ideal (%)
Baseline NUMA-GPU	49	40	0.0	49
NUMA-GPU + Page Replication	47	35	2.4	47
CARVE	6	8	3.0	6
Ideal NUMA-GPU System	0	0	0.0	0

Performance Gains: CARVE reduces slowdown to just **6%** compared to the ideal NUMA-GPU system, outperforming baseline NUMA-GPU (49% slowdown) and page replication techniques (47% slowdown).

Reduced Remote Accesses: Lowers remote memory accesses from **40%** (baseline) to **8%**, significantly reducing latency caused by inter-GPU bandwidth limitations.

Minimal Memory Overhead: It uses only **3% of GPU memory**, ensuring negligible impact on overall memory availability.

Scalability and workload-specific improvement : CARVE consistently delivers benefits across various workloads, effectively handling both read-only and read-write shared data. Workloads like **Lulesh** and **XSbench** benefit the most from reduced remote traffic, while latency-sensitive tasks like **RandAccess** see minor limitations due to RDC misses.

Paper 13: Analysis of the Security Vulnerabilities of 2.5-D and 3-D Integrated Circuits

This paper investigates the unique security challenges associated with advanced IC technologies, specifically 2.5-D and 3-D integrated circuits (ICs). These technologies, which leverage vertical stacking and interposers for enhanced performance, bring significant improvements in **device density, memory bandwidth, and energy efficiency**. However, their complex architectures and reliance on globalized supply chains introduce vulnerabilities to **hardware Trojans** and **side-channel attacks**. The paper thoroughly analyzes these vulnerabilities, classifies their mechanisms, and discusses their implications for modern IC design.

Key Contributions of the Paper

1. Security Vulnerabilities in 2.5-D and 3-D ICs

- The paper identifies the unique security risks in 2.5-D and 3-D ICs compared to traditional 2-D ICs:
 - **Testing Limitations:** Limited access to internal components, such as Through-Silicon Vias (TSVs) and vertical interconnects, makes it challenging to detect malicious modifications.
 - **Process, Voltage, and Temperature (PVT) Variations:** Stacked designs exhibit significant variations, masking the detection of security breaches like hardware Trojans.
 - **Increased Leakage:** Higher device density and thermal gradients exacerbate side-channel signal leakage, making these ICs more vulnerable to exploitation.

1. Hardware Trojans

A **Hardware Trojan** is a malicious modification intentionally inserted into an integrated circuit (IC) during design, manufacturing, or assembly. These modifications compromise the functionality, security, or reliability of the IC. Hardware Trojans are particularly dangerous because they are embedded directly into the physical hardware, making them challenging to detect and mitigate.

Placement:

- **TSV/Interposer Trojans:** Exploit the TSVs or interposer structure to inject malicious payloads or disrupt IC operation.
- **Cross-Tier Trojans:** Spread across multiple tiers in 3-D ICs, where trigger mechanisms and payloads are distributed, making detection harder.
- **Middle-Tier Trojans:** Target middle layers of stacked ICs, which are thermally and physically less accessible during testing.

Trigger Mechanisms: **Thermal Activation:** Use heat accumulation in stacked ICs to trigger Trojan activation. For instance, increased heat can alter electrical parameters, causing transitions to unspecified states or glitches. **Non-Thermal Activation:** Triggered by logic or signal properties (e.g., specific logic states, resistance, or capacitance changes in TSVs).

Payloads:

- **Information Leakage:** Trojans transmit sensitive data, such as encryption keys or IP, to external attackers.
- **Denial of Service (DoS):** Cause system failures by short-circuiting TSVs, inducing timing violations, or degrading IC reliability.

Classification:

- **Thermal-Activated Trojans:** Exploit heat dissipation challenges in stacked ICs as a trigger mechanism.
- **Non-Thermal Trojans:** Leverage electrical properties like resistance and capacitance of TSVs to disrupt signal integrity or induce timing violations.
- **Cross-Tier Trojans:** Trojans are distributed across multiple tiers, requiring components in different layers to work together to activate the malicious payload.

Detection: Limited testing access and inability to simulate real-world conditions (e.g., thermal buildup) make Trojan detection difficult.

Mitigation: Split manufacturing and heat dissipation techniques provide partial solutions but fail to fully address Trojan risks.

2. Side-Channel Attacks

Side-Channel Attack (SCA) exploits unintentional physical signals generated by an IC during normal operation to extract sensitive information. Unlike direct attacks on algorithms, SCAs use indirect information like power consumption, timing delays, thermal profiles, or electromagnetic (EM) emissions to infer the inner workings of the circuit.

- **Attack Types:**

- **Thermal Side-Channel Attacks (TSC):** Use IR radiation from ICs to infer activity patterns or encryption keys. Attackers exploit heat propagation across tiers or modules.
- **Power-Based Side-Channel Attacks (SCA):** Analyze variations in power consumption to extract sensitive data. Advanced attacks like Correlation Power Analysis (CPA) use statistical methods to refine key searches.
- **Electromagnetic Attacks:** Analyze EM radiation to reconstruct internal computations.

2.5-D/3-D ICs are particularly vulnerable due to their stacked architectures, which amplify thermal and power-based side-channel signals.

3. Vulnerability Models and Threat Scenarios

Trojans:

- **Untrusted Facilities:** Threats arise from untrusted foundries, interconnect fabricators, or assembly facilities that can introduce malicious components.
- **TSV/Interposer Risks:**
 - Trojans targeting TSVs can affect both signal and power delivery, introducing timing delays or increased aging in devices.
 - Signal-TSVs can be exploited for information leakage through electromagnetic radiation.
- **Cross-Tier Trojans:**
 - Trojan payloads spread across tiers are harder to detect due to testing challenges.
 - Specific attacks include snooping critical information or altering encryption functionality via TSV manipulation.

SCAs:

- **Amplified Signals:** Stacked designs increase thermal and power signals, making leakage easier to exploit.
- **Non-Invasive:** SCAs require no physical tampering, relying on external observations.

Impact

- Information leakage, including cryptographic keys and sensitive operations.
- Performance degradation caused by additional measures to prevent leakage.

4. Countermeasure Limitations

- While some countermeasures, like split manufacturing and enhanced functional testing, have been proposed, the paper highlights their inefficiencies:
 - **Split Manufacturing:** Attackers can exploit structured results from placement and routing algorithms to bypass security.
 - **Thermal Management Challenges:** Poor thermal properties in 3-D ICs make it difficult to implement effective defenses against thermal-activated Trojans or thermal side-channel attacks.

Paper 14: Present and Future, Challenges of High Bandwith Memory (HBM)

High Bandwidth Memory (HBM) is a cutting-edge memory technology that has become essential in addressing the growing demand for high-performance and energy-efficient solutions in modern computing. This paper explores the challenges and advancements of HBM technology, highlighting its role in GPUs, AI accelerators, and high-performance computing (HPC) applications.

Present Challenges of HBM

- **Thermal Management:** One of the primary challenges facing HBM technology is thermal management. The stacked die structure of HBM results in higher power density, which can lead to increased heat generation. This heat concentration can potentially impact the performance and reliability of HBM devices, necessitating advanced cooling solutions.
- **Cost and Complexity:** HBM implementation involves complex manufacturing processes, including through-silicon vias (TSVs) and die stacking. These intricate procedures contribute to higher production costs compared to traditional memory technologies. The increased complexity also poses challenges in terms of yield and overall manufacturing efficiency.
- **Interposer Integration:** The integration of HBM with other components, particularly through silicon interposers, presents significant technical challenges. Ensuring proper signal integrity and power delivery across the interposer requires careful design considerations and advanced packaging techniques.

Future Challenges and Opportunities

Scaling and Capacity As the demand for memory bandwidth continues to grow, scaling HBM to higher capacities while maintaining performance and efficiency will be a critical challenge. Future iterations of HBM will need to address this by exploring new materials, optimizing die stacking techniques, and improving overall system integration.

Power Efficiency: While HBM offers improved energy efficiency compared to traditional memory technologies, further optimizations will be necessary to meet the increasing demands of power-constrained applications. This may involve innovations in circuit design, materials science, and system-level power management strategies.

Ecosystem Development: The widespread adoption of HBM technology will require a robust ecosystem of tools, design methodologies, and standards. Developing and refining this ecosystem to support HBM integration across various applications and platforms will be an ongoing challenge for the industry.

Reliability and Longevity: As HBM finds its way into more diverse applications, ensuring long-term reliability and durability under various operating conditions will become increasingly important. This may involve advancements in materials, packaging technologies, and testing methodologies.

Feature	HBM2	HBM3	HBM3E
Bandwidth	256 GB/s	819 GB/s	1.18 TB/s
Capacity (12-Stack)	16 GB	24 GB	36 GB
Thermal Dissipation	Standard	Improved	10% Higher
Power Savings	-	Moderate	50%

The table showcases the remarkable advancements in High Bandwidth Memory (HBM) technology across generations, reflecting its critical role in addressing the growing demands of high-performance computing and AI workloads.

1. **Bandwidth Improvements:**
 - HBM has achieved consistent bandwidth growth, from 256 GB/s in HBM2 to 1.18 TB/s in HBM3E, overcoming data movement bottlenecks in GPUs and AI accelerators.

2. **Capacity Scaling:**

- The increase from 16 GB to 36 GB in 12-stack configurations reflects HBM's ability to meet growing memory demands while maintaining compact form factors.

3. **Thermal Management:**

- Innovative solutions like MR-MUF packaging have improved heat dissipation by 10% in HBM3E, addressing the thermal challenges of stacked DRAM architectures.

4. **Power Efficiency:**

- HBM3E integrates advanced logic dies to achieve a 50% reduction in power consumption, supporting energy-efficient AI workloads and mitigating heat generation.

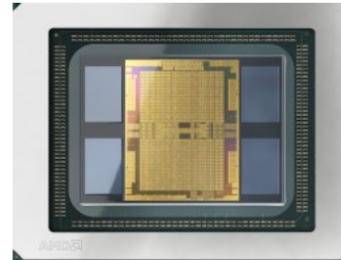
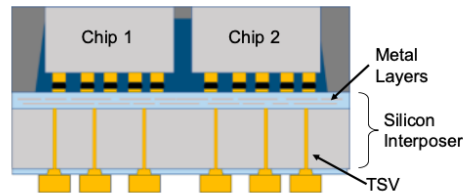
HBM's evolution demonstrates a remarkable ability to meet the growing demands of high-performance and data-intensive computing workloads. Despite challenges like thermal management, manufacturing complexity, and power delivery, advancements in bandwidth, capacity, and efficiency reflect sustained innovation. As the industry transitions to future generations, integrating near-memory and in-memory processing will further cement HBM's role in next-generation AI and HPC systems.

Paper 15: The Next Era for Chiplet Innovation

The paper "The Next Era for Chiplet Innovation" by Gabriel H. Loh and Raja Swaminathan from AMD explores the emerging technologies and challenges in advanced semiconductor packaging and integration. As Moore's Law slows down and silicon manufacturing costs increase, chiplet-based architectures have emerged as a promising solution to continue improving computational capabilities.

What Are Chiplets?

- Chiplets are smaller modular dies designed to be integrated into a single package using advanced interconnects and stacking technologies.



(a)

(b)

Figure 19 - (a) Cross-section schematic of two chips stacked on a passive silicon interposer, (b) an example processor die with four memory modules all on a silicon interposer [15]

- They address silicon yield issues and rising costs by replacing large monolithic SoCs with multiple smaller, specialized dies.

Why Chiplets?

- The slowdown of Moore's Law and escalating silicon costs necessitate a shift toward modular, heterogeneous architectures.
- Chiplets enable flexibility in die manufacturing, allowing different components to be produced on different technology nodes (e.g., logic on 7nm and IO on 12nm).

Current Approaches in Chiplet Integration

- **2D Multi-Chip Modules (MCMs):**
 - Partition functionality across smaller dies to improve yield and reduce cost.
 - Trade-offs include reduced interconnect bandwidth and increased latency.

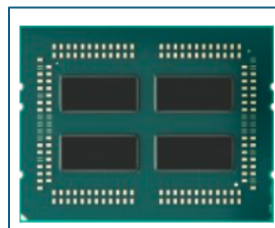


Figure 20 - an example four-die MCM. [15]

- **2.5D Silicon Interposers:** 2.5D interposers enable high-density interconnections between chiplets by using passive silicon interposers with multiple metal layers. These interposers provide bandwidths far exceeding traditional multi-chip modules (MCMs).
- Example: The **AMD Instinct MI100**, utilizing 2.5D interposers, achieves **1.2 TB/s bandwidth** by integrating GPU compute dies with four HBM2 memory modules in a single package.
- **Impact:** This architecture significantly improves data transfer rates for applications like AI inference and training, where high memory bandwidth is crucial.

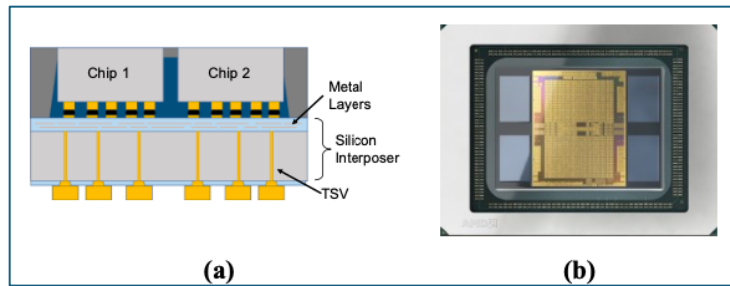


Figure 21 - (a) Cross-section schematic of two chips stacked on a passive silicon interposer, (b) an example processor die with four memory modules all on a silicon interposer.[15]

- **Silicon Bridges (Elevated Fanout Bridges - EFB):**

- Provide a cost-effective alternative to interposers by connecting smaller chiplets using high-density bridges.
- Example: AMD Instinct™ MI200 connects two GPU dies and eight memory modules using silicon bridges.

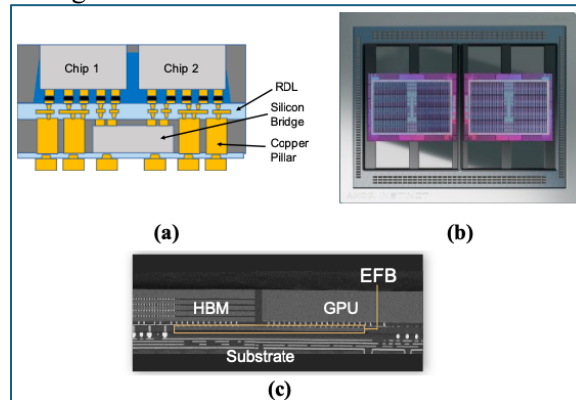


Figure 22- (a) Cross-section schematic of two chips connected via a silicon bridge, (b) using silicon bridges to connect eight memory modules to two processor dies, and (c) a cross-sectional view of one of the bridge assemblies.[16]

- **3D Stacking:** 3D stacking as a transformative advancement, allowing dies to be stacked vertically using technologies like hybrid bonding. This technique not only increases integration density but also enhances memory performance by reducing inter-die communication latency.
 - Microbumps and hybrid bonding are used to vertically stack active dies
 - The **AMD V-Cache™** leverages hybrid bonding to stack additional L3 cache dies on top of CPUs, increasing **cache capacity by 3x**. This innovation reduces latency and boosts memory bandwidth, making it ideal for data-intensive tasks.
 - 3D stacking is particularly advantageous for workloads requiring low-latency, high-throughput memory access, such as HPC and cloud computing.

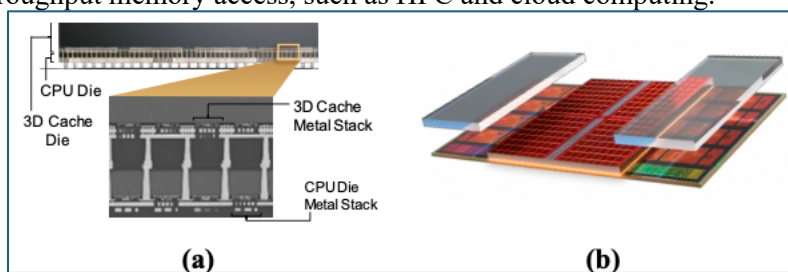


Figure 23 - (a) Cross-section micrographs of a cache die 3D hybrid bonded to a CPU chiplet,

(b) another view of a cache die stacked on a CPU chiplet with additional filler silicon [15]

Power and Thermal Metrics

Energy Efficiency: Chiplet-based designs optimize energy efficiency by reducing the need for long interconnects between functional units. By localizing data movement within smaller dies, these designs minimize power loss.

- **Advanced Interconnects:** Technologies like **hybrid bonding** and **high-density TSVs** further improve energy efficiency by enabling low-power communication between chiplets.
- **Impact:** This is critical for applications like edge computing and mobile devices, where power constraints are paramount.

Thermal Challenges: While chiplet architectures enhance performance, they also introduce significant thermal challenges due to the proximity of dies and increased power density in stacked designs.

- **Hybrid Bonding Benefits:** The paper discusses how hybrid bonding improves thermal pathways between dies by reducing inter-die gaps. However, these improvements require advanced workload scheduling to prevent hotspots.
- **Impact:** Addressing thermal challenges is essential for ensuring the reliability and longevity of 3D-stacked chiplet architectures.

Cost and Yield Improvements One of the major advantages of chiplets is their ability to improve manufacturing yield. By partitioning large monolithic designs into smaller chiplets, defects in one die do not render the entire package unusable.

- **Node Optimization:** The paper discusses how different components can be fabricated using different process nodes. For example, logic dies can use cutting-edge 7nm technology, while IO dies can use cost-effective 12nm processes, reducing overall costs.
- **Impact:** This modular approach not only lowers manufacturing costs but also accelerates time-to-market for new designs, making it particularly appealing for cost-sensitive industries.

Standardization and Scalability: The paper emphasizes the need for standardized interconnect protocols to enable scalability and interoperability across vendors. The **Universal Chiplet Interconnect Express (UCIe)** is highlighted as a significant step toward creating a unified ecosystem for chiplets.

- **UCIe Benefits:** By providing a common framework, UCIe allows vendors to mix and match chiplets from different manufacturers, fostering innovation and competition.
- **Impact:** Standardization is key to scaling chiplet architectures across a wide range of applications, from consumer electronics to large-scale data centers.

Chiplet-based architectures represent a paradigm shift in semiconductor design, enabling continued scaling beyond the physical limitations of traditional silicon technology. By addressing current challenges and fostering ecosystem development, chiplets are poised to drive advancements in GPUs, AI accelerators, and high-performance computing systems. The success of this approach will depend on innovations in interconnects, packaging, and software-hardware co-design, ensuring that chiplet architectures can meet the growing demands of future workloads.

Discussion

The reviewed papers collectively address critical challenges in GPU microarchitecture, focusing on improving **memory bandwidth**, **energy efficiency**, and **resource utilization**. This section compares and evaluates the methodologies and findings under three thematic groups:

Memory Hierarchy Optimization

1. SIMT Efficiency and Control Flow Divergence
2. Energy Efficiency and Architectural Scalability

The key findings are synthesized into three focus areas.

1. Memory Hierarchy Optimization: HBM-enabled GPUs are pivotal in addressing memory bandwidth challenges, but their limited capacity necessitates innovative solutions. Double-buffering techniques effectively mitigate the oversubscription issue by dynamically overlapping memory transfers with computation. This reduces memory consumption, exemplified in AlexNet training, where memory usage dropped from 4.35GB to 3.52GB, achieving a $1.63\times$ speedup. Moreover, techniques like **cache-aware memory hierarchies** prioritize frequently accessed data, reducing memory traffic and improving energy efficiency without sacrificing performance.

HBM's Role in GPU Performance:

- HBM significantly improves throughput, with **HBM3E achieving 1.18 TB/s**, a substantial leap from HBM2's 256 GB/s.
- Double-buffering pipelines leverage this bandwidth to reduce latency. For instance, in AlexNet training, memory consumption dropped from **4.35GB to 3.52GB**, achieving a $1.63\times$ speedup by overlapping computation and memory transfers.

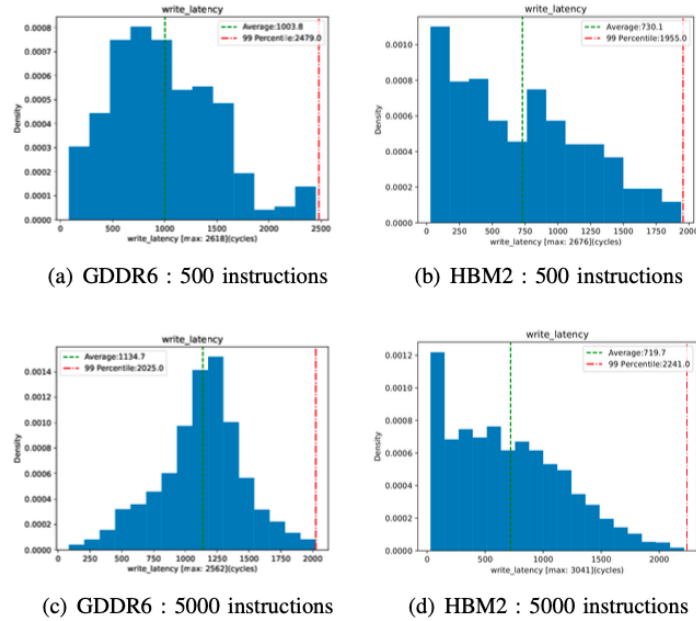


Figure 24 - Evaluation of GDDR6 and HBM across different number of write instructions [9]

This figure above evaluates the performance of **GDDR6** and **HBM2** across varying numbers of write instructions, showcasing the latency improvements achieved with HBM2.

- **Latency Reduction:** HBM2 consistently demonstrates lower write latency compared to GDDR6, with an average latency of **730.1 cycles (500 instructions)** versus **1003.8 cycles** for GDDR6.
- **Scalability:** As the number of instructions increases to 5000, HBM2 maintains relatively stable latency, whereas GDDR6 exhibits higher variability and increased write latency.
- These results highlight HBM's superiority in handling high-throughput workloads, particularly for applications demanding frequent write operations like CNN training or BFS traversal.
- The stability of HBM2 latency across larger workloads underscores its suitability for scaling GPU architectures, addressing memory bottlenecks more effectively than traditional memory technologies.

Locality-Aware Hierarchies:

- Adaptive caching mechanisms dynamically adjust cache usage based on workload patterns. For example, **LAMAR's granularity adaptation** minimizes over-fetching, improving bandwidth utilization while maintaining low energy costs.

Graph Workloads and Irregular Memory Patterns:

- Breadth-First Search (BFS) workloads demonstrate the limitations of current memory hierarchies. Techniques like warp-centric programming models improve access coalescence but are still constrained by HBM's capacity.

Future Directions

1. Hybrid Memory Solutions

To address the scalability and cost challenges associated with HBM, integrating **Hybrid Memory Systems (HMS)** offers a promising direction. These systems combine **High Bandwidth Memory (HBM)** with **Storage-Class Memory (SCM)**, such as 3D XPoint or NVRAM, to achieve a balance between performance, capacity, and cost. We discussed SCM in paper 7 [7]

1. Advantages of Hybrid Memory:

- **Scalability:** SCM provides higher storage capacities at a lower cost compared to HBM, which can complement HBM's high-speed but limited-capacity design.
- **Energy Efficiency:** SCM consumes less power per bit of storage, making it an excellent candidate for energy-aware GPU designs.
- **Workload Adaptability:** Hybrid memory can dynamically allocate high-throughput data to HBM while offloading less time-critical data to SCM.

2. Implementation Challenges:

- **Data Placement:** Efficiently managing where data resides (HBM vs. SCM) is critical to maintaining low latency.
- **Memory Coherence:** Ensuring data consistency across memory layers requires innovative coherence protocols.
- **Access Granularity:** SCM's larger access granularity can introduce inefficiencies if not optimized for GPU workloads.

3. Research Opportunities:

- Develop **machine learning-based memory management algorithms** that dynamically allocate data across hybrid systems based on access patterns and workload demands.
- Explore **hardware-software co-designs** for seamless integration of SCM into GPU memory hierarchies.

2. Advanced Cooling Solutions for HBM

One of the most significant barriers to wider adoption of HBM is its **thermal management requirements [16]**. We remember discussing in Prof. Kaeli's class how cooling technology turned things around for IBM when it was no longer able to bear the cost.

The stacked die architecture of HBM increases power density, leading to potential overheating and performance throttling.

1. **Emerging Cooling Techniques:**

- **Micro-Channel Cooling:** Integrates liquid cooling directly into the interposer or silicon substrate to dissipate heat more effectively.
- **Embedded Phase-Change Materials (PCM):** Uses materials like paraffin wax that absorb heat as they transition between solid and liquid states, providing consistent thermal control.
- **Thermal Conductive Interposers:** Replaces traditional interposers with materials designed to conduct heat away from the memory stack more efficiently.

2. **Recent Advancements:**

- **Multi-Row Redistribution Layers (MR-MUF):** Demonstrated in HBM3E, MR-MUF enhances heat dissipation by optimizing the flow of heat away from high-power regions.
- **Graphene-Coated Interposers:** Experimental designs show that graphene interposers can reduce thermal resistance by up to 50%.

3. **Research Opportunities:**

- Develop **predictive thermal modeling algorithms** to identify hotspots dynamically and allocate tasks accordingly. Already there are dedicated teams doing thermal analysis in ASIC companies, but if given priority like real-time analysis like emulation framework will give us much better direction.
- Investigate **AI-driven thermal management systems** that optimize cooling resource allocation in real-time based on workload intensity.

By combining **Hybrid Memory Systems (HMS)** with **Advanced Cooling Solutions**, the next generation of GPUs can achieve unprecedented levels of scalability and efficiency. These innovations are critical not only for traditional GPU workloads like AI and scientific computing but also for emerging applications such as autonomous systems, edge computing, and real-time analytics.

2. SIMT Efficiency and Control Flow Divergence

Control flow divergence is one of the most significant challenges in **SIMT (Single Instruction, Multiple Threads)** architectures. Divergence occurs when threads within a warp follow different execution paths, leading to underutilized resources and wasted computational cycles. Addressing this issue is crucial for improving GPU performance across diverse workloads.

Macro-Level View

At the core, techniques addressing divergence aim to:

1. **Minimize Idle Cycles:** By reassigning threads dynamically to new warps or compacting active threads, efficiency is improved.
2. **Enhance Warp Utilization:** Scheduling strategies like thread compaction and warp prioritization improve resource utilization.
3. **Reduce Memory Traffic:** Efficient memory coalescence helps alleviate performance bottlenecks associated with divergence-heavy workloads.

Several approaches have emerged:

1. **Thread Block Compaction (TBC):**

- Dynamically groups threads based on their execution paths within a block.
- Improves warp utilization by eliminating inactive threads, achieving up to a **22% speedup** over traditional per-warp stacks.

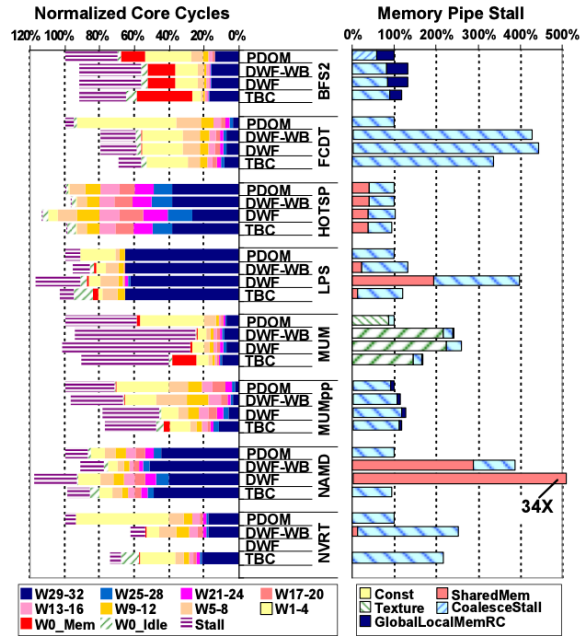
2. **Dynamic Warp Formation (DWF):**

- Reassigns threads to new warps dynamically during execution.
- Particularly effective for nested control flows but introduces scheduling overhead.

3. **Hybrid Techniques:**

- Combining thread compaction with warp prioritization policies (e.g., round-robin or age-based scheduling) balances fairness and efficiency.

Aspect	Description	Experimental Results
Reconvergence Efficiency	The block-wide stack mechanism in TBC accelerates reconvergence by sharing control points across warps.	Instruction stalls reduced by 30% in nested workloads compared to baseline per-warp stacks.
Warp Utilization and Scheduling	Age-based scheduling prioritizes older warps for fairness and improved throughput.	Throughput improved by 15% using age-based scheduling compared to round-robin policies.
BFS Traversal Performance	Hybrid traversal strategies with warp-centric mapping enhance load balancing and performance.	Clock cycles reduced by 36% for BFS traversal; hybrid traversal achieved a 2.5× speedup over thread-centric approaches.
Normalized IPC Gains	TBC achieves significant IPC improvements over baseline and DWF methods across divergence-heavy workloads.	TBC showed 22% higher normalized IPC compared to per-warp reconvergence stacks.



(a) Core Cycle Breakdown (b) Memory Pipe Stall Breakdown
Figure 25: Detail performance data of TBC and DWF relative to baseline (PDOM) for the DIVG benchmark set.[1]

Core Cycle Breakdown (Left): Shows how core cycles are distributed across active, idle, and stalled states for TBC, DWF, and PDOM. TBC reduces **idle cycles** significantly, meaning fewer threads are waiting or stalled compared to DWF and PDOM. More cycles are spent in active execution (blue and purple bars), indicating better warp utilization and less wasted computational power.

Memory Pipe Stall Breakdown (Right): Highlights memory stalls caused by operations like coalesced memory access or shared memory contention. TBC minimizes memory stalls, especially in divergence-heavy workloads like BFS, where efficient memory handling is crucial. For challenging workloads like NAMD, TBC achieves a **34× reduction in memory stalls**, showing its ability to handle irregular memory patterns effectively.

Future Directions for SIMT Efficiency

1. **AI-Driven Scheduling:** Future intelligent schedulers should dynamically adapt to workload patterns, optimizing warp compaction and thread prioritization in real time. For example, frameworks like **HOSHMANT**, which utilize Recurrent Neural Networks (RNNs) for dynamic task scheduling, highlight the potential of AI in adapting to heterogeneous resources and evolving workloads. By analyzing real-time performance metrics, such AI-driven systems could predict thread behaviors and allocate GPU resources more efficiently, reducing idle cycles and enhancing throughput.[7]
2. **Hardware-Software Co-Design:** Future GPU architectures must adopt advanced co-design methodologies to address resource bottlenecks and improve scalability. Embedding techniques like **Thread Block Compaction (TBC)** directly into hardware would reduce scheduling overhead, streamline thread management, and enhance real-time scheduling capabilities. Insights from multi-threaded architectures, such as those based on **RISC-V multicore systems [18]**, demonstrate the potential of integrating hybrid thread management and cache mechanisms at the hardware level. These approaches enable efficient handling of divergent workloads, reduce latency, and improve parallel performance. For instance, studies have shown that co-designing thread scheduling and execution pipelines can achieve speedups of up to **400%** while maintaining minimal resource overhead. Applying such principles to GPU architectures would create scalable solutions capable of supporting increasingly complex workloads.
3. **Application-Specific Optimization:** Adapting divergence reduction strategies to specific workloads, like decision trees or irregular graph traversals, is key to improving GPU efficiency. By tailoring architectural optimizations to fit the unique requirements of each application, GPUs can better handle challenging tasks. For example, techniques such as **coordinated warp scheduling** and **cache prioritization [19]** have proven effective in speeding up critical warps, particularly in graph analytics and AI inference workloads.

These approaches focus on dynamically managing GPU scheduling and memory access to avoid redundant operations and improve performance. Research shows that such workload-specific strategies can achieve performance gains of up to **3×** in tasks involving irregular memory and compute patterns. Applying these principles to GPU architectures would make them more efficient and versatile, enabling them to support a wider range of applications while keeping energy consumption low.

3. Energy Efficiency and Architectural Scalability

Introduction to the Challenge

As GPUs continue to scale to meet the demands of data-intensive applications, energy efficiency becomes a critical factor for ensuring scalability, particularly in environments like data centers and embedded systems. The inherently high parallelism of GPUs drives performance gains but also significantly increases power consumption, creating challenges in cooling, energy costs, and system sustainability.

1. **Locality-Aware Memory Hierarchies**
 - **Description:** Efficiently managing data locality within memory hierarchies reduces unnecessary memory traffic and optimizes cache usage.
 - **Impact:**
 - Energy consumption is reduced by up to **40%** by leveraging spatial and temporal locality for better cache line fetching and reducing redundant memory accesses.
 - **Example:** Studies have shown that adaptive memory hierarchies that prioritize frequently accessed data can lower energy costs significantly in workloads like matrix multiplication and AI inference. [3]

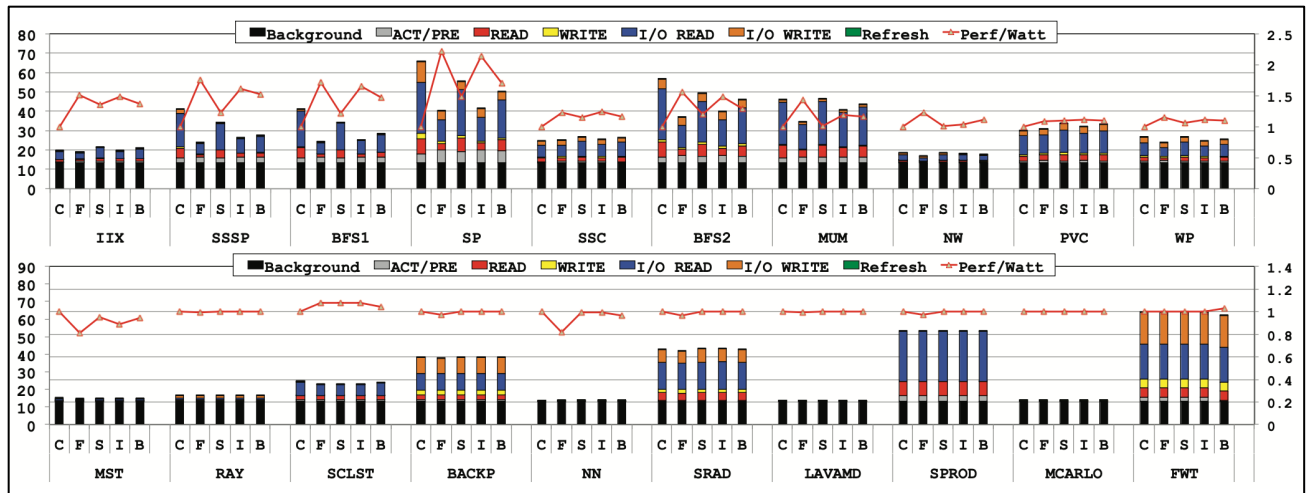


Figure 26: A breakdown of DRAM power (Watt, left axis) and the corresponding Perf/Watt (normalized to CG-only, right axis). [3]

This figure separates power usage into key categories, such as **READ**, **WRITE**, **I/O traffic**, **refresh**, and **background power**, offering insight into how energy is distributed within the memory system. The red line graph highlights the normalized **Perf/Watt** metric (right axis), showcasing the efficiency of optimized memory hierarchies compared to traditional designs.

Key observations include:

- Significant reductions in **READ/WRITE power** for workloads like **BFS1** and **SP**, demonstrating the benefits of locality-aware memory optimizations.
- Improved **Perf/Watt** for workloads with high data locality, indicating energy savings alongside sustained performance.

2. Dynamic Resource Allocation

- **Description:** Dynamically adjusts GPU resources based on workload demands to minimize idle power consumption without sacrificing throughput.
- **Impact:**
 - Achieves up to **2.9× energy efficiency improvements**, particularly in real-time workloads with variable compute requirements. [3]
- **Example:** In gaming and real-time analytics, dynamic allocation ensures power is concentrated where it's needed, reducing energy waste while maintaining performance.

Logic-Based HBM Integration and Chiplet Synergy

Logic-based High Bandwidth Memory (HBM) integration and chiplet-based GPU architectures together address critical energy efficiency and scalability challenges. By embedding logic layers directly within the HBM stack and leveraging modular chiplet designs, this approach achieves localized data processing and optimal resource management, reducing both energy consumption and latency.

- **Energy Efficiency Through Proximity:** The proximity of logic layers within HBM stacks eliminates unnecessary data traversal across memory hierarchies, cutting energy costs by 50% compared to traditional designs. Chiplet-based architectures further enhance this by integrating specialized memory controllers and logic units near HBM, ensuring workload-specific bandwidth optimization.
 - For example, **AI inference tasks** have shown up to **40% faster data access** in such configurations, particularly in deep learning models like GPT-based architectures. For instance, Micron's HBM3 Gen2 memory delivers over 1.2 TB/s bandwidth, significantly

reducing training times for models such as GPT-4 and improving infrastructure efficiency for AI inference. [21]

- Additionally, Micron's HBM3E technology offers a 30% reduction in power consumption compared to competitors, further optimizing performance and energy efficiency in AI applications. [22] These advancements underscore HBM's critical role in accelerating data access and processing in AI workloads.
- **Thermal Management:** Advanced innovations like Molded Underfill (MR-MUF) packaging and hybrid bonding reduce thermal resistance, allowing for the safe scaling of HBM stacks (e.g., HBM3E). Chiplet-based designs complement this by spreading heat generation across smaller units, improving cooling efficiency and ensuring thermal stability in dense configurations. [14]

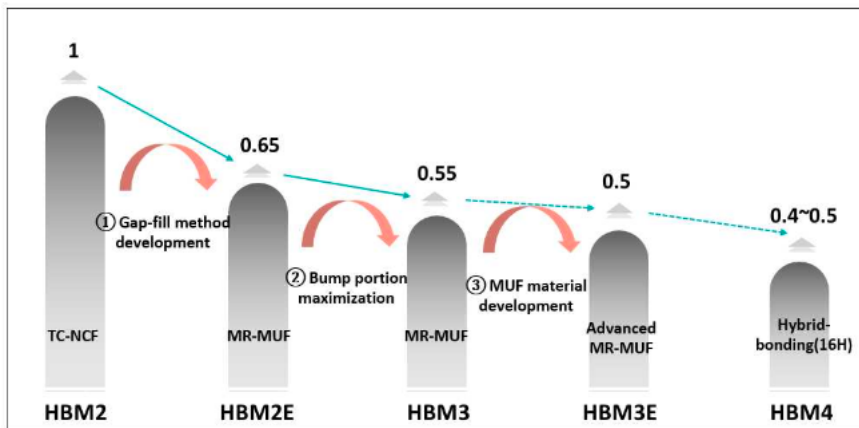


Figure 27: Thermal resistance according to MR-MUF process improvement

The figure above illustrates the improvements in thermal resistance achieved through advanced MR-MUF packaging technology, specifically designed for HBM3E. By enhancing heat dissipation properties and integrating additional Through-Silicon Vias (TSVs), this approach addresses critical challenges in thermal management. The advancements enable scalable architectures by minimizing overheating risks, ensuring higher energy efficiency for data-intensive applications

Chiplet and HBM Synergy:

- **Localized Processing:** Combining chiplets with HBM minimizes latency and energy usage by processing data closer to memory.
- **Scalability:** Modular chiplets enable customized configurations for diverse workloads, allowing for optimized designs in AI inference and graph analytics.

Performance Insights

- The combination of logic-based HBM and chiplet-based designs significantly enhances energy efficiency and scalability.
- Studies show a **40% reduction in energy consumption** for workloads leveraging locality-aware hierarchies and optimized chiplet-based configurations.
- The incorporation of **hybrid memory stacks** (e.g., HBM with non-volatile memory) provides an additional layer of energy savings during idle cycles while maintaining high throughput.

Future Directions :

Hybrid Memory Stacks

The combination of **High Bandwidth Memory (HBM)** with newer **non-volatile memory (NVM)** technologies, such as **ReRAM** or **3D XPoint**, offers a practical solution for balancing performance and

energy efficiency in GPUs. This integration can help reduce idle power consumption by up to 30% while sustaining high data throughput during demanding workloads. Future developments should explore better memory management techniques, such as algorithms that dynamically allocate data between HBM and NVM, to maximize performance while minimizing energy usage.

Edge Computing and AI Applications

Chiplet-based GPU designs are particularly suited for applications in **edge computing**, such as **autonomous vehicles** and **IoT devices**, where low latency and efficient power usage are essential. Integrating **HBM with chiplets** could enable real-time data processing capabilities tailored to these specific needs. Further research into workload management and energy-efficient architecture designs could improve the scalability and adaptability of GPUs for these rapidly evolving applications.

Conclusion:

This project has explored key advancements and challenges in GPU microarchitecture, focusing on critical areas such as memory bandwidth, energy efficiency, and SIMT utilization. The integration of technologies like High Bandwidth Memory (HBM), chiplet-based designs, and workload-specific optimizations has shown significant promise in addressing current limitations and paving the way for future developments.

Moving forward, GPU microarchitecture will require continued innovation and collaboration across multiple disciplines. Future GPUs will need to incorporate smarter workload management systems, leveraging AI to optimize performance and energy use in real time. While several studies focus on HBM's energy efficiency, few explore its integration with emerging non-volatile memory technologies like ReRAM. Combining HBM with emerging memory technologies, such as non-volatile memory (e.g., ReRAM), will help strike a balance between capacity and energy efficiency. Similarly, refining chiplet-based designs will open opportunities to create highly customized architectures that cater to specific workloads, such as graph analytics and real-time AI inference. Tailored solutions for reducing divergence in irregular workloads will further improve efficiency and broaden the scope of GPU applications. The findings and discussions in this study address some of today's most pressing GPU challenges while highlighting future directions for growth. As GPUs continue to evolve, they will remain critical for handling the demands of data-intensive workloads, AI-driven applications, and emerging fields like autonomous systems and edge computing. By building on the advancements discussed here, GPUs are well-positioned to deliver better performance and energy efficiency while staying adaptable to the changing needs of modern computing.

Citations:

1. M. Zhu, Y. Zhuo, C. Wang, W. Chen and Y. Xie, "**Performance Evaluation and Optimization of HBM-Enabled GPU for Data-Intensive Applications**," in IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 26, no. 5, pp. 831-840, May 2018, doi: 10.1109/TVLSI.2018.2791442.
2. W. W. L. Fung and T. M. Aamodt, "**Thread block compaction for efficient SIMT control flow**," **2011 IEEE 17th International Symposium on High Performance Computer Architecture**, San Antonio, TX, USA, 2011, pp. 25-36, doi: 10.1109/HPCA.2011.5749714. keywords: {Instruction sets;Graphics processing unit;Compaction;Hardware;Kernel;Pipelines;Random access memory},
3. M. Rhu, M. Sullivan, J. Leng and M. Erez, "**A locality-aware memory hierarchy for energy-efficient GPU architectures**," 2013 46th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO), Davis, CA, USA, 2013, pp. 86-98. keywords: {Graphics processing units;Bandwidth;Random access memory;Memory management;Throughput;Instruction sets;GPU;SIMD;SMT;Fine-Grained Memory Access;Irregular Memory Access Patterns;Adaptive Granularity Memory}, keywords: {Graphics processing units;Bandwidth;Memory management;Training;Neural networks;Power demand;Random access memory;Artificial neural networks;concurrent computing;memory management},
4. G. Koo, Y. Oh, W. W. Ro and M. Annavaram, "Access pattern-aware cache management for improving data utilization in GPU," 2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA), Toronto, ON, Canada, 2017, pp. 307-319, doi: 10.1145/3079856.3080239. keywords: {Graphics processing units;Instruction sets;Benchmark testing;Hardware;Computer architecture;Kernel;Monitoring;GPGPU;cache management;memory access patterns}
5. A. Bakhoda, G. L. Yuan, W. W. L. Fung, H. Wong and T. M. Aamodt, "**Analyzing CUDA workloads using a detailed GPU simulator**," **2009 IEEE International Symposium on Performance Analysis of Systems and Software**, Boston, MA, USA, 2009, pp. 163-174, doi: 10.1109/ISPASS.2009.4919648. keywords: {Analytical models;Yarn;Graphics;Parallel processing;Microarchitecture;Hardware;Process design;Concurrent computing;Parallel programming;Computational modeling},
6. S. Moon, S. Nam, J. Son and S. Srikant, "**System-level Power Integrity Optimization Based on High-Density Capacitors for enabling HPC/AI applications**," 2020 4th IEEE Electron Devices Technology & Manufacturing Conference (EDTM), Penang, Malaysia, 2020, pp. 1-4, doi: 10.1109/EDTM47692.2020.9117919. keywords: {Silicon devices;Voltage fluctuations;Preforms;Capacitors;Capacitance;MIM capacitors;Manufacturing;HPC/AI;high-power applications;power integrity;power delivery network;decoupling capacitor;system-level design optimization},
7. J. Hong, S. Cho, G. Park, W. Yang, Y. -H. Gong and G. Kim, "**Bandwidth-Effective DRAM Cache for GPU s with Storage-Class Memory**," 2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA), Edinburgh, United Kingdom, 2024, pp. 139-155, doi: 10.1109/HPCA57654.2024.00021. keywords: {Performance evaluation;Costs;Power demand;Instruction sets;Memory management;Random access memory;Graphics processing units;Graphics processing unit (GPU);Storage-Class Memory (SCM);DRAM cache;3D-stacked memory;Cache bypassing},
8. Jishen Zhao and Yuan Xie. 2012. Optimizing bandwidth and power of graphics memory with hybrid memory technologies and adaptive data migration. In Proceedings of the International Conference on Computer-Aided Design (ICCAD '12). Association for Computing Machinery, New York, NY, USA, 81–87. <https://doi.org/10.1145/2429384.2429400>
9. A. K. Kabat, S. Pandey and V. T. Gopalakrishnan, "Performance evaluation of High Bandwidth Memory for HPC Workloads," 2022 IEEE 35th International System-on-Chip Conference

- (SOCC), Belfast, United Kingdom, 2022, pp. 1-6, doi: 10.1109/SOCC56010.2022.9908071. keywords: {Three-dimensional displays;Memory management;Random access memory;Bandwidth;Benchmark testing;Energy efficiency;Silicon;High Bandwidth Memory;GDDR5;GDDR6;HPC Workloads},
10. S. Hong, T. Oguntebi and K. Olukotun, "Efficient Parallel Graph Exploration on Multi-Core CPU and GPU," 2011 International Conference on Parallel Architectures and Compilation Techniques, Galveston, TX, USA, 2011, pp. 78-88, doi: 10.1109/PACT.2011.14. keywords: {Graphics processing unit;Arrays;Bandwidth;Parallel processing;Proposals;Optimization;Graph;BFS;GPU;Multi-Core CPU},
 11. H. Peters, M. Köper and N. Luttenberger, "Efficiently Using a CUDA-enabled GPU as Shared Resource," 2010 10th IEEE International Conference on Computer and Information Technology, Bradford, UK, 2010, pp. 1122-1127, doi: 10.1109/CIT.2010.204. keywords: {Kernel;Graphics processing unit;Sorting;Instruction sets;Memory management;Synchronization;Bismuth},
 12. V. Young, A. Jaleel, E. Bolotin, E. Ebrahimi, D. Nellans and O. Villa, "Combining HW/SW Mechanisms to Improve NUMA Performance of Multi-GPU Systems," 2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO), Fukuoka, Japan, 2018, pp. 339-351, doi: 10.1109/MICRO.2018.00035. keywords: {Graphics processing units;Bandwidth;Hardware;Coherence;Random access memory;Transistors;GPU;Multi-GPU;Memory;NUMA;HBM;DRAM-Cache;Coherence;Page-Migration;Page-Replication},
 13. V. V. Rao, A. Sasan and I. Savidis, "Analysis of the Security Vulnerabilities of 2.5-D and 3-D Integrated Circuits," 2022 23rd International Symposium on Quality Electronic Design (ISQED), Santa Clara, CA, USA, 2022, pp. 1-7, doi: 10.1109/ISQED54688.2022.9806155. keywords: {Integrated circuits;Solid modeling;Side-channel attacks;Intellectual property;Hardware;Trojan horses;Security},
 14. K. Kim and M. -j. Park, "Present and Future, Challenges of High Bandwidth Memory (HBM)," 2024 IEEE International Memory Workshop (IMW), Seoul, Korea, Republic of, 2024, pp. 1-4, doi: 10.1109/IMW59701.2024.10536972. keywords: {Heating systems;Industries;Scalability;High performance computing;Memory management;Bandwidth;Pressing;HBM;Stacked DRAM;TSV;2.5D SiP;High bandwidth DRAM;MR MUF;AI Memory},
 15. G. H. Loh and R. Swaminathan, "The Next Era for Chiplet Innovation," *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, Antwerp, Belgium, 2023, pp. 1-6, doi: 10.23919/DATE56975.2023.10137172. keywords: {Industries;Semiconductor device modeling;Technological innovation;Moore's Law;Stacking;Computer architecture;Trademarks;chiplets;integration;stacking},
 16. K. Son et al., "Design and Analysis of Thermal Transmission Line based Embedded Cooling Structures for High Bandwidth Memory Module and 2.5D/3D ICs," 2020 IEEE Electrical Design of Advanced Packaging and Systems (EDAPS), Shenzhen, China, 2020, pp. 1-3, doi: 10.1109/EDAPS50281.2020.9312897. keywords: {Through-silicon vias;Cooling;System-on-chip;Integrated circuits;Thermal management;Heat transfer;Power transmission lines;High bandwidth memory;Junction temperature;Signal integrity;Thermal integrity;Thermal Transmission Line},
 17. M. Bidollahkhani, A. K. Sharma and J. M. Kunkel, "HOSHMAND: Accelerated AI-Driven Scheduler Emulating Conventional Task Distribution Techniques for Cloud Workloads," 2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC), Osaka, Japan, 2024, pp. 2313-2320, doi: 10.1109/COMPSAC61105.2024.00372. keywords: {Cloud computing;Adaptation models;Job shop scheduling;Recurrent neural networks;Processor scheduling;Computational modeling;Dynamic scheduling;High-Performance Computing;AI-driven Scheduling;Recurrent Neural Network;Dynamic Job Scheduling},
 18. B. Kieu-Do-Nguyen et al., "Hardware Software Co-design for Multi-threaded Computation on RISC-V-based Multicore System," in *IEEE Access*, doi: 10.1109/ACCESS.2024.3505940.

keywords: {Multicore processing;Instruction sets;Software;Hardware;Multithreading;Memory management;Resource management;Read only memory;Prefetching;Internet of Things;RISC-V;Multithread;Multicore;Task scheduling;Hardware-Software Co-design},

19. S. -Y. Lee, A. Arunkumar and C. -J. Wu, "CAWA: Coordinated warp scheduling and Cache Prioritization for critical warp acceleration of GPGPU workloads," 2015 ACM/IEEE 42nd Annual International Symposium on Computer Architecture (ISCA), Portland, OR, USA, 2015, pp. 515-527, doi: 10.1145/2749469.2750418. keywords: {Instruction sets;Round robin;Acceleration},
20. H. Liu, A. Aljbri, J. Song, J. Jiang and C. Hua, "Research advances on AI-powered thermal management for data centers," in Tsinghua Science and Technology, vol. 27, no. 2, pp. 303-314, April 2022, doi: 10.26599/TST.2021.9010019.
keywords: {Cooling;Computational modeling;Data centers;Atmospheric modeling;Thermal management;Data models;Biological system modeling;data center;thermal management;artificial intelligence;heuristic;metaheuristic},
21. [1] "Micron Delivers Industry's Fastest, Highest Capacity HBM to Advance Generative AI Innovation," TechPowerUp, [Online]. Available: <https://www.techpowerup.com/311709/micron-delivers-industrys-fastest-highest-capacity-hbm-to-advance-generative-ai-innovation>. [Accessed: Dec. 2, 2024].
22. [2] "Micron's HBM3E: Powering the Future of AI with High Bandwidth Memory," Micron, [Online]. Available: <https://www.micron.com/about/blog/applications/ai/microns-hbm3e-powering-the-future-of-ai-with-high-bandwidth-memory>. [Accessed: Dec. 2, 2024].